

HARNESS ENGINEERING & PRODUCTION AGENT — PRACTICE COURSE

하네스 엔지니어링 기반 프로덕션 수준 에이전트 개발

비결정적 LLM을 결정론적 비즈니스 프로세스로 통제하는 실전 엔지니어링
추론 조율 · 컨텍스트 설계 · 도구 통제 · 평가 하네스 · 거버넌스 총망라

INSTRUCTOR

김형욱 강사

CONTACT

hukim@artiasolution.com

COURSE OUTLINE

전체 교육 목차 (Table of Contents)

01 하네스 엔지니어링 기반 Agentic AI 개발

p. 3

02 추론 방식 및 멀티에이전트 설계

p. 16

03 컨텍스트 엔지니어링 및 메모리 아키텍처

p. 33

04 도구 엔지니어링 및 커스텀 MCP 구축

p. 55

05 AI Agent 평가 체계와 신뢰성 엔지니어링

p. 70

06 운영 거버넌스, 권한 통제 및 모니터링

p. 83

MODULE 01

하네스 엔지니어링 기반 Agentic AI 개발

에이전트의 지능은 모델이 아닌 하네스에서 나온다.
실전 비즈니스 환경을 견디는 아키텍처 방법론 개요

하네스의 탄생 배경

모델의 한계와 하네스(Harness)의 필요성

 Model (원시 LLM)

추론 & 의사결정

사용자 의도 해석, 작업 계획 수립, 도구 호출 판단

 **단독 한계:** 단발성 텍스트 출력(One-turn), 무상태성(Stateless), 외부 환경 직접 실행 불가

↓ 모델을 오케스트레이션하여 과업을 완수시키는 실행 인프라 ↓

 Harness (에이전트 런타임 & 제어 프레임워크)

실행·상태·거버넌스 인프라

 **상태·컨텍스트**
메모리 영속화·주입 **도구 오케스트레이션**
API·DB 대행 실행 **자율 루프 제어**
과업 완료까지 ReAct **거버넌스·가드레일**
권한 승인·루프 차단**AI Agent = Model (추론 및 의사결정) + Harness (오케스트레이션 및 실행 인프라)**

"모델이 '무엇을 할지 판단'하면, 하네스가 '실제 실행하고 안전하게 제어'합니다."

원시 LLM만으로는 복합 비즈니스 과업을 완수할 수 없습니다

모델은 뛰어난 언어 추론 지능을 갖췄지만, 본질적으로 단발성 텍스트 변환 함수입니다. 자체적으로 이전 작업 상태를 영속 보존하지 못하며(Stateless), 외부 시스템에 직접 접근해 명령을 수행할 수 없습니다.

하네스(Harness)란? 모델을 오케스트레이션하여 과업을 완수시키는 실행 시스템

- **상태 및 컨텍스트 관리 (State & Context):** 필요한 데이터와 작업 히스토리를 영속화하고 모델에 선별 주입하여 문맥 유지.
- **도구 오케스트레이션 (Tool Execution):** 모델의 호출 의도를 받아 파일 I/O, API, DB 쿼리를 격리 환경에서 안전하게 대행 실행.
- **자율 추론 루프 제어 (Autonomous Loop Control):** 목표 과업이 완수될 때까지 [추론 ➔ 도구 실행 ➔ 결과 관찰] 사이클을 지속 구동.
- **거버넌스 및 안전 가드레일 (Governance & Guardrails):** 무한 루프 탐지·차단, 비정상 출력 교정 및 고위험 작업 권한 검증(HITL).

 **개념 팁:** '하네스(Harness)'는 본래 거친 동력을 목적에 맞게 결속·제어하여 유용한 일(Work)로 변환하는 장치를 뜻합니다.

하네스 아키텍처 개요

에이전틱 하네스의 정의 (Model vs Harness)

Model (추론 엔진)

- 자연어를 이해하고 추론하는 지능을 공급 (토큰 단위 예측)
- Gemini, GPT, Claude, LLaMA 등 — 언제든지 교체 가능한 "추론 엔진" 역할
- 자체적인 상태 보존, 장기 기억, 외부 리소스 직접 제어 능력 부재

Harness (실행 및 제어 프레임워크)

에이전트 워크플로우(e.g. ReAct)의 동작 환경 고도화

비결정적 LLM 출력을 결정론적 비즈니스 프로세스로 변환하는 중재 계층

- **프롬프트 조립 (컨텍스트 라이프사이클)**: 시스템 프롬프트 합성 및 캐시 친화적 프롬프트 배치
- **컨텍스트 관리**: 메모리 읽기/쓰기, 대화/작업 히스토리 영속 보존 및 실시간 Compaction 필터링
- **도구 라우팅 (도구 샌드박스)**: 격리 샌드박스 내부에서 안전하게 실행 및 에러 복구
- **평가 및 디버깅 (평가 하네스)**: 행동 궤적 및 최종 상태 검증, 회귀 테스트 및 힐 클라이밍 튜닝
- **오류 복구 & 권한 통제 (거버넌스)**: Allow/Ask/Deny 3단 권한 체크 및 실시간 무한 루프 감지기 가동

에이전트 패러다임

AI 에이전트의 정의와 핵심 추론 패턴

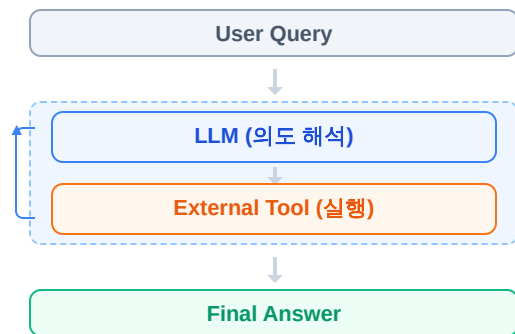
자율적 인지와 추론 기반의 시스템

단순히 고정 규칙(Rule-based)에만 반응하는 자동화 코드를 넘어, 상황을 파악하고 최적의 행동 경로를 스스로 생성하여 과업을 해결합니다.

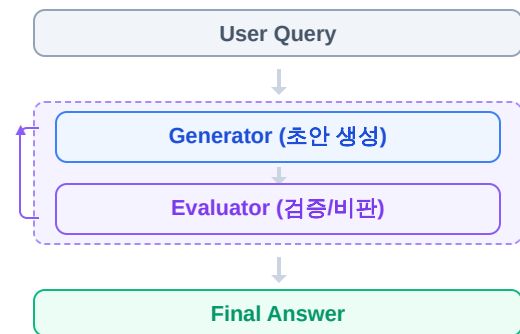
목표와 복잡도에 최적화된 4대 핵심 추론 패턴

- **Tool Use (도구 사용)**: LLM이 필요한 도구를 판단하고 실행 결과를 피드백하는 기본 루프.
- **ReAct (Reasoning + Acting)**: 사고와 행동을 반복 수렴해 나가는 범용 패턴.
- **Reflection (성찰)**: 결과물을 자체 검증 및 회고하여 스스로 교정하는 루프.
- **Planning (계획 수립)**: 목표 달성을 위한 서브 태스크(DAG 등)를 사전 기획해 제어.

Tool Use (도구 사용)

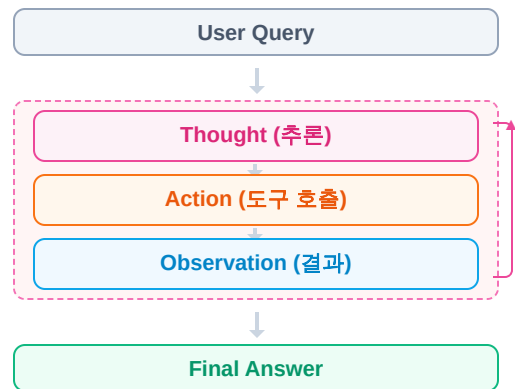


Reflection (성찰)

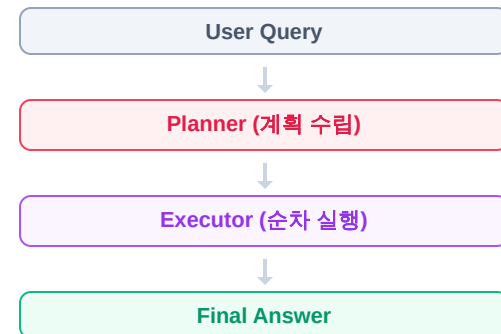


대표 패턴

ReAct (추론 + 행동)



Plan & Execute (기획)



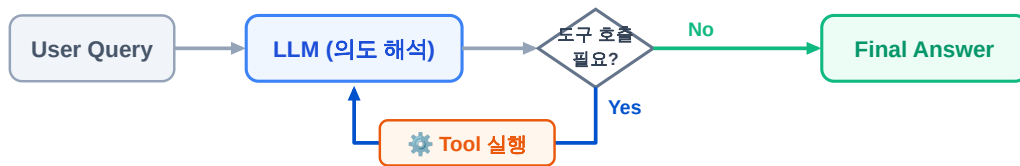
에이전트 패러다임

4대 핵심 추론 패턴의 상세 워크플로우 비교

💡 AI 에이전트 추론 패턴의 핵심: 단순히 LLM을 일회성으로 호출하는 방식을 넘어, **자율적인 의사결정(Decision)**과 **컨텍스트 업데이트 루프(Loop-back)**를 통해 비즈니스 목표를 완수합니다.

🌐 Tool Use (도구 사용)

LLM이 필요한 도구를 판단하고 실행 결과를 컨텍스트에 피드백하며 목표를 수행하는 기본 루프.



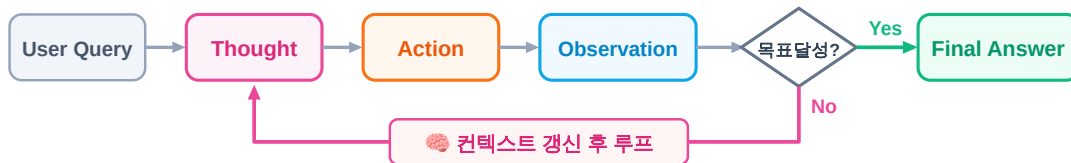
🔄 Reflection (성찰)

초안을 생성하는 Generator와 품질을 검증/평가하는 Evaluator가 피드백을 주고받는 패턴.



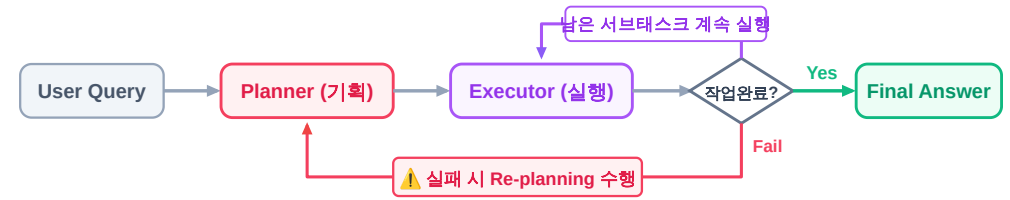
🔥 ReAct (추론 + 행동)

Thought(추론), Action(도구 선택), Observation(관찰)을 순환하며 과업을 자율 완수하는 핵심 패턴.



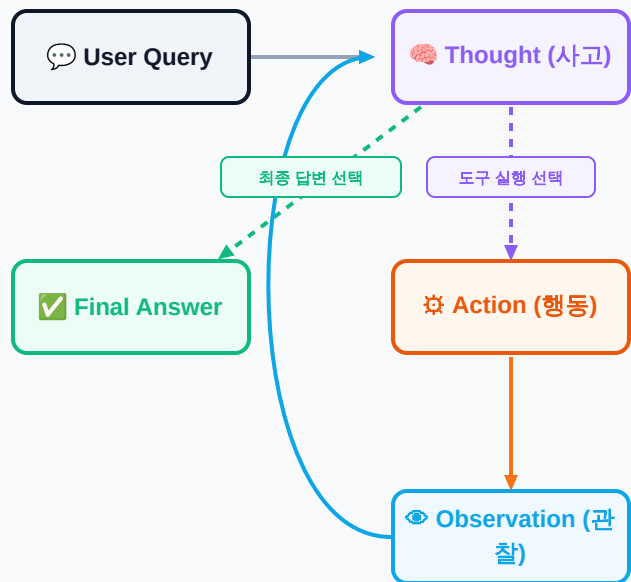
📋 Plan & Execute (계획 후 실행)

목표 달성을 위한 여러 하위 태스크를 사전 기획하고, 실패 시 Re-planning을 수행하는 패턴.



에이전트 작동 원리

ReAct (Reasoning + Acting) 에이전트 워크플로우



ReAct Execution Trajectory Trace (Compact)

QUESTION

"서울 기온에 5를 곱한 결과는?"

THOUGHT 1

서울의 날씨 검색 도구(`search_weather`)를 호출한다.

ACTION 1

call: `search_weather(location="Seoul")`

OBSERVATION 1

`{"temp": "24°C", "condition": "Sunny"}`

THOUGHT 2

온도 24에 5를 곱하기 위해 계산기 도구(`calculator`)를 호출한다.

ACTION 2

call: `calculator(expression="24 * 5")`

OBSERVATION 2

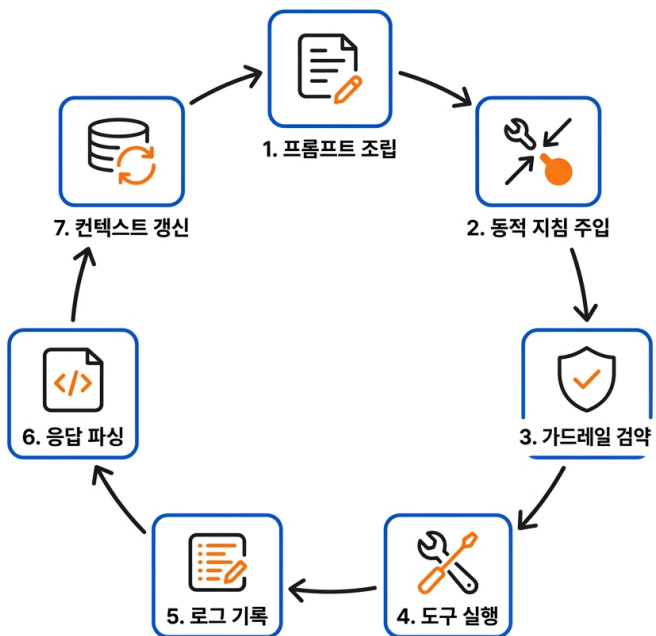
120

FINAL ANSWER

"서울 기온(24°C) x 5 = 120입니다."

에이전트 작동 원리

에이전트 하네스의 7단계 실행 루프

**1. 프롬프트 조립 (Prompt Assembly)**

규칙 지침, 대화 기록, 작업 상태를 캐시 친화적으로 취합하여 컨텍스트 조립.

2. 동적 지침 주입 (Dynamic Instruction Injection)

작업에 필요한 동적 메타데이터, 환경 변수 및 관련 가이드라인 주입.

3. 가드레일 검증 (Guardrail Verification)

실행 전 프롬프트 주입 방지, 탈옥 탐지 및 안전 가이드라인 검사.

4. 도구 실행 (Tool Execution)

허가 완료된 명령을 격리 샌드박스 내부에서 결정론적으로 구동.

5. 로그 기록 (Audit Logging)

수행된 도구 호출 경로와 사용 토큰, 지연 시간 메타데이터 적재.

6. 응답 파싱 (Response Parsing)

도구 실행으로 반환된 출력값을 가로채 유효성 체크 및 포매팅 수행.

7. 컨텍스트 갱신 (Context Update)

정리된 작업 변경 사항을 작업 메모리에 누적하여 다음 루프 준비.

하네스 아키텍처 개요

하네스 아키텍처: 3대 축, 4대 패턴, 4대 중재 장치



하네스 핵심 3대 축 (인프라)

- **메모리 (Memory)**: 단기 대화 상태와 시맨틱 장기 메모리를 관리하여 에이전트의 작업 문맥(Context)을 연속적으로 보존합니다.
- **스킬 (Skills)**: 사전에 설계되거나 동적으로 등록된 도구(APIs) 및 코드 실행 파일들을 에이전트가 호출할 수 있도록 적재합니다.
- **프로토콜 (Protocols)**: 에이전트와 도구, 외부 플랫폼 간의 데이터 형식 및 상호작용 표준 계약(ACI) 규격을 제공합니다.

자율 조율 & 추론 4대 패턴 (지능)

- **ReAct (추론+행동)**: 사고(Thought), 행동(Action), 관찰(Observation) 순환 루프.
- **Reflection (자가성찰)**: Generator와 Evaluator 간의 성찰 및 자가보정 루프.
- **Planning (계획수립)**: 복잡과업 분할(Decomposition) 및 실패 시 재계획(Re-planning).
- **Multi-Agent (협업조율)**: Handoff 및 Agent-as-Tool 기반의 유기적 조율.

안전 실행 4대 중재 장치 (거버넌스)

- **샌드박스 (Sandbox)**: 코드 및 도구 실행을 호스트 서버와 격리하는 안전 환경.
- **관측성 (Observability)**: 실행 궤적(Trace)을 실시간 적재하여 사후 감사(Audit) 지원.
- **승인 게이트 (Approval)**: 중요 동작 및 예산 초과 시 인간 승인(HITL)을 연동.
- **가드레일 (Guardrail)**: 프롬프트 주입 공격 및 유해 입출력을 실시간 탐지/차단.

하네스 설계 프레임워크

하네스 설계를 위한 7가지 결정 캔버스 (7 Decisions Canvas)

결정 차원	선택안 A (모델 신뢰 / 얇은 하네스)	선택안 B (코드 통제 / 두꺼운 하네스)	실무 적용 기준 및 가이드라인
1. 에이전트 수	단일 에이전트 (Single Agent)	멀티 에이전트 (Multi-Agent)	등록 도구 30개 초과 혹은 관심 도메인 분리 필요 시 분화
2. 추론 전략	ReAct (즉각 대응/자율 루프)	Plan-and-Execute (계획 후 실행)	순서 종속성이 뚜렷하고 병렬 처리가 필요할 시 DAG 구조 사용
3. 컨텍스트 전략	풍부한 맥락 (Rich Context)	강한 압축 (Strong Compaction)	Attention 저하 방지를 위해 불필요 로그 요약 및 Clipping
4. 검증 방식	추론적 검증 (LLM-as-a-judge)	계산적 검증 (Linter / Schema)	구조 및 사양 에러는 계산식으로, 의미론적 교정은 추론식 검사
5. 권한 설계	허용적 권한 (allow-first)	제한적 권한 (deny-first)	배포, 파일 삭제, 외부 결제 등 고위험 명령은 반드시 승인 게이트
6. 도구 범위	전체 도구 상시 노출	동적 최소 노출 (JIT Tool Loading)	도구 혼동 방지를 위해 Task 매칭 도구들만 동적으로 주입
7. 하네스 두께	얇음 (Thin - 모델 위임)	두꺼움 (Thick - 강한 통제)	모델의 추론 안정도와 요구하는 보안 엄격도에 비례하여 조정

안티패턴 방지

세 가지 안티패턴(Anti-patterns)



01. 도구는 많을수록 좋다?

선택지 과다로 인한 성능 저하 및 어려움 증가

↓ 역발상 솔루션

✓ 필요한 최소 도구만 노출

- ✓ Vercel v0 사례: 불필요한 도구를 제거하고 파일시스템 단위로 간소화하여 성공률 80% => 100% 및 속도 3.5배 달성
- ✓ 도구 설명 자체도 귀중한 컨텍스트 리소스입니다. 태스크 맞춤 레이어 설계가 중요합니다.



02. 매 턴마다 깊게 추론하면 좋다?

무한 루프, 높은 지연 시간과 비용 폭증

↓ 역발상 솔루션

✓ 필요한 만큼만 계획 후 실행

- ✓ LLMCompiler 연구: 순서가 뚜렷한 반복 업무는 ReAct 대신 계획-실행(DAG)으로 병렬화 수행
- ✓ 지연시간 개선 최대 3.7배, 비용 감소 최대 6.7배 성과 증명



03. 권한이 허용적일수록 빠르다?

고위험 액션 실수로 되돌릴 수 없는 영구 피해 초래

↓ 역발상 솔루션

✓ 최소 권한과 마찰 설계

- ✓ 안전 장치인 '승인 마찰'은 불편함이 아니라 필수 생명선입니다.
- ✓ Allow / Ask / Deny 설계: 고위험 작업(배포, 결제 등)은 반드시 검증 게이트와 인간 개입 통제

하네스 설계 결정 워크시트 작성 실습

실습 목표 및 대상 선정

- **목표:** 실습 워크시트(부록 C)를 사용하여 비즈니스 타겟 에이전트의 구체적 아키텍처 Blueprint 수립
- **대상:** 각 팀별로 자사 비즈니스 시나리오 중 LLM 에이전트화할 수 있는 핵심 업무 선정 (예: 자동 정산 시스템, 코드 자율 리뷰 에이전트)

워크시트 작성 3단계 프로세스

- **1단계 (인터페이스 설정):** 접근해야 할 리소스(파일/DB/API)의 쓰기, 삭제 한계 규정
- **2단계 (7 Decisions Canvas):** 에이전트 개수, 추론 전략, 메모리 압축 차원의 근거 작성
- **3단계 (오작동 방지 가이드라인):** 허용 불가한 Deny 임계치 및 비상 차단 (Shutdown) 조건 설계

하네스 설계 관점

하네스 엔지니어링의 두 가지 관점



앱 유저 / 서비스 관점

"Agentic Experience"

사용자가 마주하는 완성된 에이전트 환경. 하네스는 제품 내부에 숨겨져 있으며, 심리스한 UX를 제공하는 것이 목표입니다.

- 예: Claude Code, Antigravity
- 특징: 도구 연결, 자동화된 루프, 즉각적 피드백

</> 개발자 / 프레임워크 관점

"Building Scaffolding"

에이전트를 구축하기 위한 설계 도구. 오케스트레이션 로직과 안전장치를 직접 코드로 구현하는 과정입니다.

- 예: LangChain, LangGraph, CrewAI
- 특징: 상태 관리, 명시적 라우팅, 검증 노드 설계

요약 및 정리

1교시 핵심 요약 및 정리

에이전트 지능은 모델이 공급하지만, 실제 실행 통제는 하네스가 담당합니다

- 자연어 추론은 교체 가능한 엔진인 LLM에 위임하되, 프롬프트 조립, 컨텍스트 관리(메모리 읽기/쓰기 및 영속 보존), 도구 라우팅, 실행 권한 통제는 하네스가 처리하여 비결정적인 출력을 신뢰 가능한 비즈니스 프로세스로 변환합니다.

단방향 호출 극복을 위해 4대 추론 패턴 및 자율 피드백 루프를 구동합니다

- Tool Use, Reflection, ReAct, Plan-then-Execute 등 의사결정 분기(Decision Node)와 순환 루프(Loop-back)가 포함된 4대 핵심 패턴을 구현하여, 실패 시 스스로 교정하고 복잡한 업무를 주도적으로 해결합니다.

하네스 설계는 인프라(3대 축) · 지능(4대 패턴) · 거버넌스(4대 중재)의 결합체입니다

- 메모리/스킬/프로토콜의 핵심 축 위에서, 4대 추론 엔진을 엮고, 샌드박스/관측성/승인 게이트/가드레일의 4대 거버넌스 중재 장치를 맞물려 완전한 아키텍처 Blueprint를 완성해야 합니다.

3대 설계 안티패턴을 배제하고 두 가지 엔지니어링 관점을 조율합니다

- 도구 최소 노출(Vercel v0), 계획-실행 병렬화(LLMCompiler), 안전 승인 마찰(Allow/Ask/Deny)로 성능과 안전을 도모하고, 사용자 신뢰(UX)와 개발자 코드 격리(DX) 관점을 통합 설계해야 합니다.

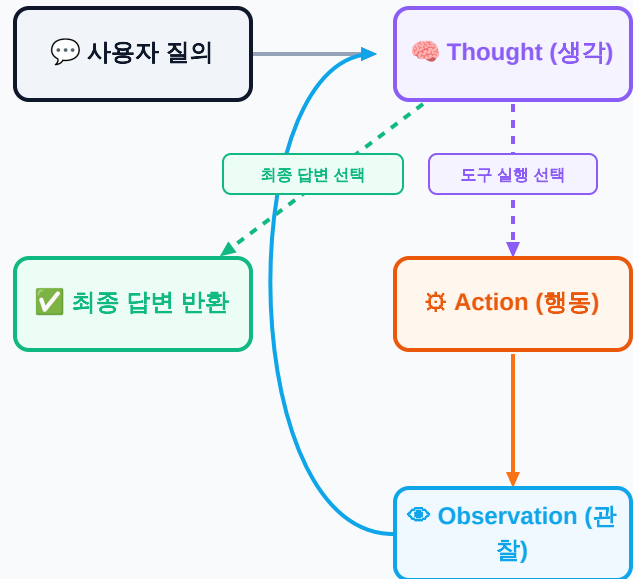
MODULE 02

추론 방식 및 멀티에이전트 설계

단순한 ReAct의 한계를 짚어보고,
계획(Planning), 자가 복구(Self-Recovery), 그리고 오케스트레이션 아키텍처를 다룹니다.

안티패턴 분석

Naive ReAct 패턴의 메커니즘과 4대 한계



1. 재시도 낭비 (Retry Waste)

에러 원인이 수정 불가능한 구조임에도 "재시도하겠다"며 20회 이상 LLM을 무차별 호출하여 API 비용이 폭증합니다.

2. 컨텍스트 오염 (Context Bloat)

도구 실행 오류 시 수백 줄의 raw stack trace가 그대로 쌓이며, 핵심 시스템 지침을 망각하는 Attention Decay를 초래합니다.

3. 결정론적 제어 부재 (No Control)

유료 결제, 파일 파괴 등 고위험 액션을 인간 승인(HITL) 가드레일 없이 에이전트의 독단 추론만으로 실행해 버립니다.

4. 확장성 한계 (Scalability Limit)

수많은 도구 명세가 하나의 컨텍스트 창에 상시 적재되어 모델의 올바른 도구 선택 성공률이 기하급수적으로 저하됩니다.

실행 흐름 통제

실행 하네스의 해법: ReAct 한계 극복을 위한 4대 조율 장치

추론 제어권을 모델(LLM)에 100% 위임하는 구조는 프로덕션에서 매우 위험합니다.

하네스(Harness)는 원시적인 ReAct 루프 외부에 자리 잡고, 비결정론적 오작동을 제어하여 시스템 안정성을 보장하기 위해 4대 조율 메커니즘을 코드 단에서 강제합니다.

1. 계획 수립 (Planning)

➡ 재시도 낭비 및 컨텍스트 오염 방지

매 턴 즉흥적인 추론 대신, 전체 과업 실행 단계(Task List/DAG)를 선제적 기획하고 실패 시 재계획(Re-planning) 루프를 돌며 통제합니다.

2. 자가 복구 (Self-Recovery)

➡ 에러 우회 / 장애 격리 / 서킷 브레이커

외부 API 장애나 도구 실행 에러가 터져도 에이전트가 뻗지 않고 작업을 완주할 수 있도록 하네스 차원의 3중 런타임 방어막을 가동합니다.

3. 가드레일 및 HITL

➡ 결정론적 제어권 확보

시스템 파괴나 대규모 결제 등 고위험 명령 실행 직전, 하네스가 코드로 권한 필터를 작동하며 인간 승인(HITL) 게이트를 강제합니다.

4. 멀티에이전트 협업

➡ 단일 컨텍스트 확장성 극복

단일 컨텍스트에 수많은 도구를 다 올리지 않고, 서브태스크별로 도구의 노출 범위와 역할을 격리한 독립적 에이전트들을 조율합니다.

계획 기반 추론

계획(Planning) 패턴의 3대 구현 아키텍처

1. 모델 자율형 계획

Model-Driven / CoT

사용자 질의 (User Query)

LLM 자체 추론/CoT 단계
(Reasoning / CoT)

최종 답변 (Final Answer)

2. 상태 관리형 계획

Explicit Prompt-State

사용자 질의 (User Query)

명시적 To-do 리스트
프롬프트 주입 (task.md)도구 실행 및 피드백 →
To-do 동적 업데이트 (갱신 루프)

최종 답변 (Final Answer)

3. 프로세스 통제형 계획

Explicit Process-Level (DAG)

사용자 질의 (User Query)

Planner LLM 호출
(실행 계획/DAG 수립)

Re-planning 루프

Executor 런타임 가동
(도구 병렬/순차 실행)

최종 답변 (Final Answer)

계획 기반 추론

계획 수립 방식별 특징 및 트레이드오프

비교 차원	모델 자율형 계획 (Model-Driven / CoT)	상태 관리형 계획 (Todo/Blackboard)	프로세스 통제형 계획 (DAG/Plan-then-Execute)
작동 원리	LLM 자체의 추론 경로(CoT 또는 내장 사고 토큰)에 의존해 계획 수립	프롬프트/컨텍스트 상태에 To-Do 리스트를 명시하고 매 턴 동적 업데이트	Planner가 실행 계획(DAG)을 수립하되, 실행 중 실패/예외 시 피드백 기반 동적 재계획(Re-planning) 루프 가동
지연 시간 (Latency)	낮음 ~ 보통 (단순 작업 시 빠른 응답. 단, o1/thinking 모델은 내부 추론 토큰 분량에 비례)	높음 (매 턴 계획 갱신 및 도구 호출을 순차 반복하므로 대화 턴 수에 비례)	보통 ~ 높음 (다단계 API 직렬 호출 오버헤드 존재. 단, 서브태스크 병렬 실행 시 일부 지연 상쇄)
흐름 제어력 (Control)	거의 없음 (사용자가 중간에 개입하거나 강제 통제 불가)	높음 (하네스가 매 턴 Todo 리스트를 검증 및 제어 가능)	매우 높음 (하네스가 고정된 실행 그래프를 엄격히 제어)
실무 추천 시나리오	고난도 코딩 알고리즘 해결, 복잡한 수학 연산	실시간 예외 상황과 톨 에러가 빈번해 동적 재계획이 필요한 업무	작업 단계의 의존성이 뚜렷하고 대규모 병렬 조치가 필요한 업무
대표 프레임워크	Claude Opus 4.6, o1/o3-mini, Gemini 2.5 Flash Thinking	Claude Code (task.md), BabyAGI, Blackboard 패턴 에이전트	LangGraph Plan-and-Execute, LLMCompiler

시스템 복원력 (RESILIENCE)

에이전트 루프의 자가 복구 (Self-Recovery) 3대 안전망

토이 프로젝트는 잘 될 때만 가정하지만, 실제 프로덕션에선 반드시 에러가 발생합니다. 하네스는 에이전트가 멈추거나 뺏지 않고 작업을 완주하도록 3단계 안전망을 제공합니다.



1. 에러 우회 (Failover)

API 순단 & 호출 제한 대비

특정 모델 API가 다운되거나 호출량 초과(429 Rate Limit)에 걸려도, 사용자나 에이전트 모르게 백업 모델로 즉시 갈아타서 작업을 멈추지 않고 이어갑니다.

핵심 원칙: LLM 개입 없는 투명한 우회
효과: 서비스 무중단 연속성 보장



2. 장애 격리 (Fault Isolation)

도구 예외 폭발 반경 제어

외부 DB나 API 도구 하나가 에러를 뿔었다고 에이전트 전체가 크래시 나면 안 됩니다. 예외를 안전한 '결과값 (Observation)'으로 감싸서 에이전트가 다른 방법을 찾게 만듭니다.

핵심 원칙: 에러를 결과값으로 흡수
효과: 도구 에러로 인한 프로세스 다운 방지



3. 서킷 브레이커 (Circuit Breaker)

무한 루프 & 과금 폭탄 차단

에이전트가 헛다리를 짚거나 무한 대기 상태에 빠져 반복 호출을 시작하면, 지정된 한도(호출 횟수나 예산)를 넘기는 순간 하네스가 루프를 물리적으로 끊어버립니다.

핵심 원칙: 코드 단의 물리적 강제 제동
효과: 무한 루프 및 과금 폭주 차단

시스템 복원력 (RESILIENCE)

자가 복구 3대 안전망 비교 및 운영 기준

구분	1. 에러 우회 (Failover)	2. 장애 격리 (Fault Isolation)	3. 서킷 브레이커 (Circuit Breaker)
감지 대상	모델 API 429(Rate Limit), 503 통신 중단, 미등록 모델명	도구 코드 실행 예외, 외부 네트워크 타임아웃, DB 연결 실패	동일 톨 무한 반복 호출, PENDING 상태 무한 대기, 예산 초과
하네스의 조치	지수 백오프 재시도 후 백업 모델로 즉시 전환	예외 스택을 가로채 안전한 관찰값 (Observation)으로 변환	호출 한도(`run_limit`) 도달 시 루프 강제 절단 및 종료
에이전트 인지	❌ 전혀 모름 (투명) 하네스가 뒤에서 은밀히 처리	○ 인지함 (피드백) 도구 실패 메시지를 보고 다른 대안 시도	❌ 강제 종료 에이전트 자율권 박탈 및 안전 차단
핵심 가치	서비스 연속성 (Continuity) 단일 모델 의존 탈피 및 무중단 서빙	폭발 반경 제어 (Blast Radius) 단일 도구 장애가 전체 다운으로 번지는 것 차단	자원 거버넌스 (Governance) 과금 폭탄 및 컴퓨팅 자원 낭비 원천 차단



하네스 엔지니어링의 핵심 원칙: 모델의 '자율성'만 믿고 에러 제어를 맡기면 안 됩니다. 인프라와 도구에서 발생하는 장애는 하네스가 먼저 시스템적으로 막아주고(Self-Recovery), 그 위에서 모델이 안전하게 다음 동작을 판단할 수 있도록 판을 깔아주는 것이 엔지니어링의 본질입니다.

시스템 복원력 (RESILIENCE)

하네스의 런타임 에러 대응 흐름: 자가 복구에서 인간 개입까지

🔄 단계별 에러 처리 및 에스컬레이션 흐름

[1단계] 에러 발생 (API 429, 톨 예외, 타임아웃)

▼ 하네스 즉각 개입

[2단계] 자가 복구: 에러 우회(Failover) & 장애 격리

▼ 에이전트 자율 재시도 (정상 복구 시 종료)

[3단계] 반복 실패 감지 (연속 N회 동일 장애)

▼ 서킷 브레이커 즉시 차단

[4단계] 프로세스 중단 ➔ 인간 결재선(HITL) 호출

🚨 자동 복구에서 인간 개입으로의 전환

가벼운 통신 순단이나 1회성 도구 에러는 하네스가 자동으로 감싸고 넘깁니다.

하지만 동일 에러가 3회 이상 반복되면 자율 해결이 불가능한 구조적 결함으로 판단하여, 서킷 브레이커를 작동시키고 사람에게 결재를 요청합니다.

🔑 개념 정리: 자가 복구 vs 자가 교정

- 자가 복구 (Self-Recovery / 오늘 다루는 내용): 시스템 장애, 통신 오류, 무한 루프로부터 에이전트가 죽지 않고 살아남게 하는 런타임 방어막입니다.
- 자가 교정 (Self-Correction / 2일차 평가 내용): 에이전트가 완수한 결과물을 채점하여 품질 기준에 못 미칠 때 피드백을 주고 다시 쓰게 하는 검증 파이프라인입니다.

에이전트 수 설계

단일 에이전트 (Single) vs 멀티 에이전트 (Multi-Agent) 트레이드오프

구분	단일 에이전트 (Single Agent)	멀티 에이전트 (Multi-Agent)	전환 임계점 (Transition Point)
설계 복잡도	낮음 : 하나의 작업 흐름과 상태 객체만 관리	높음 : 에이전트 간의 소통 규칙 및 조율 필요	★ 언제 전환해야 하는가? - 에이전트가 다뤄야 할 도구의 개수가 30개 이상으로 폭증할 때 - 회계 감사, 송금 대사 등 비즈니스 권한과 도메인 경계가 명확히 분리되어 정보 격리가 필수적일 때 - 단일 프롬프트 용량이 인지 임계값을 넘어 토큰 오작동 빈도가 급증할 때
컨텍스트 효율	전체 대화 이력 공유로 맥락 누설 없음 (토큰 낭비 가능성)	역할별 정보 격리로 인지 부하 감소 (토큰 절약 설계)	
통제 가능성	LLM의 자율성에 의존하므로 특정 시점 중단이 어려움	각 서브 에이전트의 입출력 가드레일이 분리되어 통제 용이	
디버깅 난이도	전체 로그 추적이 단순하나 Context Degradation 시 파악 곤란	개별 에이전트 단위로 격리 추적이 가능하므로 구조적으로 유리	

오케스트레이션 아키텍처

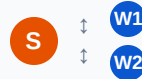
멀티에이전트 패턴

1. Network (수평적 Handoff)



- **동작 원리:** 상하 관계없이 제어권과 대화 바통을 수평적으로 양도 (예: OpenAI Swarm)
- **장점:** 라우팅이 유연하고 전환 속도가 빠름
- **단점:** 전체 실행 경로 통제가 매우 어렵고 무한 뱅뱅이 위험 존재

2. Supervisor (수직 통제)



- **동작 원리:** Supervisor 에이전트가 하위 Worker들을 도구(Agent-as-Tool)로써 수직 관리
- **장점:** 최종 결재, 가드레일, 중간 인터럽트 및 보안 통제에 극도로 유리
- **단점:** Supervisor 호출 횟수 증가로 토큰 소모 발생

3. Chain-of-Agent (파이프라인)



- **동작 원리:** 앞 단계 에이전트의 가공물이 Fan-out/Fan-in 방식으로 다음 단계로 전달
- **장점:** 흐름이 명확하고 중간 단계 병렬 처리가 용이
- **단점:** 중간 단계 실패 시 전체 파이프라인 롤백 비용이 큼

오케스트레이션 아키텍처

Handoff vs Agent-as-Tool 실행 흐름 비교

Handoff (제어권 전환) 패턴

수평적 전이 (바통 터치)

에이전트 A (시작)

▼ transfer_to_agent_b()

에이전트 B (A는 메모리 스택 소멸)

▼ transfer_to_agent_c()

에이전트 C (최종 완료 및 답변)

- 상하 주종 관계없이 대화 세션을 바통 터치식으로 인계
- 에이전트 A가 B를 호출하면 A는 대화 세션에서 완전히 제외됨
- 라우팅이 유연하지만 전체 실행 경로와 무한 루프 제어가 매우 어려움

Agent-as-Tool (Supervisor-Worker) 패턴

수직적 위임 및 반환 (중첩 루프)

Supervisor Agent

▼ invoke_sub_agent(task) [호출]

Worker Agent (서브에이전트 격리 실행)

▲ return result_data [반환 & 소멸]

Supervisor Agent (최종 취합 및 정산)

- 상위 에이전트(Supervisor)가 하위 에이전트(Worker)를 하나의 **도구(Tool)**로 사용
- 실행이 끝나면 하위 에이전트는 소멸하고, 가공된 결과 텍스트/파일만 반환
- 중앙 제어가 강제되므로 결재 승인(HITL) 및 보안 통제 적용에 극도로 유리

에이전트 간 통신 프로토콜

Agent-as-Tool (Supervisor-Worker) 패턴의 작동 메커니즘

1단계. 작업 분해 및 도구 호출

Supervisor LLM이 사용자 질의를 받아 태스크를 쪼개고, 특정 작업을 수행할 하위 에이전트를 선별합니다.

```
# Supervisor의 서브에이전트 위임 도구 등록
tools = [invoke_sub_agent, list_sub_agents]
```

- 함수 호출(Function Call) 형식으로 Worker 실행
- 실행 명령과 함께 필요한 인자(Arguments) 전달

2단계. 서브에이전트 격리 실행

호출된 **Worker** 에이전트는 전담 System Prompt와 격리된 파일시스템 메모리를 가지고 작업을 독립 실행합니다.

```
# invoke_sub_agent 도구로 격리 세션 실행
invoke_sub_agent(task_instruction,
subagent_role="scraper")
```

- 상위 Supervisor의 프롬프트 및 대화 이력 오염 완전 차단
- 개별 도구 권한을 격리하여 보안 탈옥 차단

3단계. 결과 가공 반환 및 소멸

Worker 에이전트의 작업이 완료되면, 최종 가공된 데이터나 파일 경로만 Supervisor에게 반환하고 실행 세션은 파괴됩니다.

```
# 실행 내역은 휘발, 결과만 수수
return {"status": "success", "file":
"./out.zip"}
```

- 대용량 실행 로그는 삭제하고 최종 파일/결과만 취합
- Supervisor는 정제된 결과만 전달받아 컨텍스트를 아키텍

에이전트 간 통신 프로토콜

Agent-as-Tool 패턴의 4대 프로덕션 이점

1. 컨텍스트 격리 & 정보 은닉

- **컨텍스트 격리 (Context Isolation):** 하위 에이전트가 독자적인 대화 히스토리를 가짐으로써 상위 에이전트의 대화 맥락이 오염되거나 토큰 소모량이 임계값을 넘어 폭발하는 현상을 방지합니다.
- **정보 은닉 및 추상화:** 하위 에이전트의 구체적 실행 로직이나 빌드 로그는 아래서 해소하고, 상위 에이전트에게는 **최종 가공된 결과물 파일 경로 및 성공/실패 여부만** 깔끔하게 보고합니다.

2. 안전 격리 & 쉬운 구현

- **보안 및 장애 격리:** 하위 에이전트의 에러가 전파되어 전체 그래프가 마비되는 연쇄 붕괴를 차단하며, 특정 에이전트 탈옥(Jailbreak)에 의한 피해를 격리된 샌드박스로 가둡니다.
- **쉬운 구현 (Agent-as-Tool):** 복잡한 분기 조건이나 상태 라우팅을 코딩할 필요 없이, 하위 에이전트를 Supervisor의 '**일반 도구(Tool)**' 스펙처럼 엮기만 하면 되어 생산성이 향상됩니다.

협업 프로토콜

상하위 에이전트 간의 정보 불균형 (Information Asymmetry)

Supervisor: Big Picture (전체 맥락 지배)

전체 작업 흐름과 최종 목표

- 알고 있는 정보 (Knowns):
 - 전체 태스크의 완료 조건 및 최종 인수 기준
 - 다단계 파이프라인의 장기 계획과 진행 상태
 - 사용자 세션 히스토리와 최종 의사결정 권한
- 모르는 정보 (Unknowns):
 - 하위 도구 실행 시 발생하는 런타임 세부 맥락
 - 로컬 파일시스템의 파일별 세부 라인 코드 내용
 - 도구 실행 예외의 구체적인 stack trace 로그

Sub-Agent: Small Picture (세부 실행 지배)

세부 구현 컨텍스트와 실행 결과

- 알고 있는 정보 (Knowns):
 - 자신이 맡은 부분 태스크의 톨 실행 상태
 - 로컬 환경의 실제 파일 내용 및 변경 이력
 - 컴파일러 에러 메시지와 즉각적인 런타임 피드백
- 모르는 정보 (Unknowns):
 - 이 작업이 전체 파이프라인에서 가지는 비즈니스 맥락
 - 최종 완료 단계에서 필요한 다른 도메인의 결합 지침
 - 전체 시스템 및 외부 데이터에 미치는 글로벌 영향도

⚠ 비효율적 실행의 근본 원인 (Asymmetry Wall): 상하위가 서로의 컨텍스트를 모른 채 단방향 지시만 하면, 불필요한 반복 루프 호출, 엉뚱한 방향의 작업 재시도, 혹은 컨텍스트 비대화로 인해 성능과 효율이 저하됩니다.

협업 프로토콜

정보 불균형 극복 프로토콜 (1) 컨텍스트 탐색 및 주입

1. 동적 컨텍스트 가지치기

Dynamic Context Pruning (Agentic RAG)

하위 에이전트를 기동하기 직전, 전체 프로젝트 트리과 코드베이스 임베딩을 시맨틱 검색하여 "이번 태스크 해결에 가장 의존성이 높은 파일 목록"을 선별해 컨텍스트에 주입합니다.

```
relevant_files
=
search_embeddings
(task_query)

invoke_sub_agent
(task_instruction, target_file_list=relevant_files)
```

- 하드코딩된 파일 전송 규칙 설계를 배제하여 범용성 극대화
- 불필요한 컨텍스트 유입을 막아 하위 에이전트의 입력 토큰 최적화

2. 자율적 컨텍스트 수집 권한

Self-directed Context Gathering

하위 에이전트에게 단순 실행 도구뿐만 아니라, 코드베이스 내부를 검색 및 조회 (`grep_search`, `view_file` 등)할 수 있는 탐색형 도구 권한을 부분적으로 허용합니다.

```
grep_search("import database") → 자율 검색
view_file("config/database.py") → 자율 확인
```

- 상위 에이전트가 모든 정보를 완벽하게 마운트해 줄 필요 상쇄
- 하위 에이전트가 툴 실행 과정에서 자체적으로 정보 불균형을 해결하도록 유도

협업 프로토콜

정보 불균형 극복 프로토콜 (2) 예외 처리 및 상태 공유

 3. 장애 격리 및 상위 보고

Fail-Safe Escalation

하위 에이전트는 기동 시 "해결 불가능한 치명적 예외 발생 시 무리한 재시도를 멈추고 상위에 즉시 보고하라"는 규칙을 전달받습니다.

[BLOCKER] pg_config 실행 파일 없음
[REQUIRED] 관리자 2차 승인 키 필요

- 의미 없는 재시도 낭비(Retry Waste)를 즉각 차단하여 턴 수 소모 방지
- 구조화된 장애 상태 전송 ➔ Supervisor가 계획 보정 후 재지시

 4. 공유 작업 보드

Blackboard Pattern

공통 작업 파일시스템 내에 상태 공유 텍스트(예: task.md)를 두고, 모든 에이전트가 읽고 쓰며 실시간 싱크를 맞춥니다.

- [x] API 응답 구조 추출 완료
- [/] DB 데이터 검증 로직 실행 중

- 메시지 호출 파라미터 간소화로 컨텍스트 소모 최소화
- 런타임 롤백이나 일시 중단 후 재실행 시 상태 연속성 상시 유지

 5. 프롬프트 레이어링

Prompt Layering

하위 에이전트 가동 시 공통 도구/안전 지침(Base Prompt) 위에 런타임용 임무 지침(Task Instruction)을 적층 결합하여 설계합니다.

[Base] 예외 보고는 지정 양식 고정
+ [Task] "회원가입 API 결합 수정"

- 인프라 규칙 오염 차단 및 톨 콜 응답 포맷 일관성 보장
- 개발 효율 극대화: 인프라 규칙 생략 후 순수 임무만 주입

요약 및 정리

2교시 핵심 요약 및 정리

실행 하네스의 강력한 통제가 비결정성을 제어합니다

- ReAct의 4대 한계를 이기기 위해선 에이전트단 Planner, 예산 및 턴 수 한계, 서킷 브레이커 설정이 의무화되어야 합니다.

에러 우회, 장애 격리, 서킷 브레이커로 루프의 생존성을 확보합니다

- 모델 API 장애는 에러 우회로, 도구 실행 예외는 장애 격리로, 무한 반복 호출은 서킷 브레이커로 차단하여 시스템이 멈추거나 뺏지 않는 프로덕션 복원력을 완성합니다.

Blackboard Pattern을 통한 상태 연속성을 확보합니다

- 에이전트들이 파일시스템 기반의 공유 작업 보드(task.md)를 통해 상태를 공유하도록 설계함으로써, 장시간 실행 에이전트의 맥락 유지와 소통 파라미터 크기 최적화를 동시에 실현할 수 있습니다.

MODULE 03

컨텍스트 엔지니어링 및 메모리 아키텍처

컨텍스트의 유한한 공간 제약을 극복하기 위한 5계층 프롬프트 조립, 하이브리드 RAG, L1~L4 장기 메모리 라이프사이클, 그리고 5단계 컨텍스트 컴팩션(Compaction) 전략을 다룹니다.

장기 메모리의 가치

왜 장기 메모리인가? Stateless vs Stateful 에이전트

❌ Stateless (무상태성 에이전트)

매 대화마다 기억이 초기화되어 반복적인 탐색과 비효율 유발

- 단일 세션/태스크 범위에서만 작동하여, 세션이 종료되는 즉시 작업 컨텍스트 전체가 증발합니다.
- 사용자의 선호 스타일, 과거의 의사결정, 성공/실패했던 궤적(Episodic History)을 전혀 기록하지 못합니다.
- 재가동 시 이전 정보가 전무하여 사용자에게 **기존에 했던 기초 질문을 중복해서 묻게 됩니다.**

✅ Stateful (상태성 에이전트)

사용자 피드백과 프로젝트 환경을 지속적으로 축적하는 비서형 지능 모델

- 에이전트가 실행되면서 도출된 사용자의 핵심 규칙, 스타일 피드백, 해결 성공 데이터를 메모리에 자율 기록(**WRITE**)합니다.
- 새 세션이 시작되면 영속 저장소로부터 압축 보존된 이전 맥락을 읽어와 (**READ**) 시스템 프롬프트에 즉각 결합합니다.
- 작업량이 늘어날수록 사용자 고유의 개발 스타일과 도메인 환경에 **정밀하게 최적화**됩니다.



핵심 결론: 에이전트의 작업 성공률과 안정성은 State(컨텍스트) 제어력에 비례합니다.

장기 메모리는 단순 데이터 저장소가 아니라, 컨텍스트 용량 한계를 극복하고 추론 효율성을 보장하는 핵심 하네스 제어 시스템입니다.

컨텍스트의 한계

컨텍스트 창 의 한계와 "Lost in the Middle" 현상

1. 컨텍스트 창 = 에이전트의 물리적 RAM

토큰 한계가 확장되더라도 대량의 원시 로그를 무작정 쏟아부으면 어텐션 분산(Attention Dilution)이 발생하여 추론 품질 저하, 환각 유발, 지침 누락의 직접적인 원인이 됩니다.

2. "Lost in the Middle" 현상 (Liu et al., 2023)

긴 컨텍스트를 지원하는 상용 LLM도 입력 데이터의 시작 부분(Head)과 끝부분(Tail)만 강하게 기억하며, 중간 영역(Middle)에 위치한 제약 조건과 사실 정보는 검색 회상률이 40% 이하로 급격히 저하됩니다.

3. 프롬프트 구조 설계 골든 룰 (Golden Placement)

절대 어겨서는 안 되는 시스템 헌법과 핵심 정체성은 최상단(Prefix)에 고정하고, 현재 턴에서 즉각 실행해야 할 구체적 임무와 사용자 질문은 최하단(Suffix)에 전략적으로 배치해야 합니다.

LLM 입력 위치별 주의력(Attention) 회상 곡선

▲ 최상단 (Head): 90~95% 고회상율

시스템 헌법 수칙 (System Core), 페르소나, 정렬된 도구 스펙

▼ 중간 영역 (Middle): 30~45% 주의력 급락 계곡

과거 대화 누적 로그, 도구 원시 출력 노이즈 ➡ 정보 유실 위험!

▲ 최하단 (Tail): 95%+ 고회상율

현재 턴의 사용자 프롬프트, 즉각 실행할 Task Instruction

메모리 시스템 아키텍처

메모리 통합형 에이전트 시스템 아키텍처



실시간 컨텍스트 주입 (Prompt Assembly & Read Path)

사용자 요청이 인입되면, 오케스트레이터가 도구 사양, 유사 지식(RAG), 장기 기억 스냅샷(Memory Reader)을 끌어모아 컨텍스트 조립기(Prompt Assembler)에서 단일 프롬프트로 병합하여 LLM에 공급합니다.

백그라운드 메모리 영속화 (Background Persistence & Write Path)

LLM 추론 결과가 도출되면 사용자에게 즉시 스트리밍 답변을 전송하는 동시에, 백그라운드 메모리 기록 미들웨어(Memory Writer)가 비동기 구동되어 새로운 사실과 대화 이력을 로컬 영속 저장소(state.db, 마크다운 파일)에 안전하게 저장합니다.

컨텍스트 파이프라인

하네스 수명주기 기반 컨텍스트 파이프라인

⚙ 에이전트 실행 중: 컨텍스트를 구성하는 2가지 핵심 경로

경로 1 프롬프트 조립 (사전 메모리 주입 / Static & Snapshot)

에이전트가 실행을 시작할 때, 실행 하네스가 전역 지침(AGENTS.md), 정렬된 도구 스펙, 그리고 장기 기억 스냅샷(MEMORY.md, USER.md)을 시스템 프롬프트에 사전 조립하여 일괄 주입합니다.

- 모델 추론 시작 전 100% 캐시 최적화 및 페르소나/규칙 환경 확립

경로 2 도구를 통한 자료/메모리 읽기·쓰기 (런타임 동적 로딩 / JIT Retrieval)

작업 도중 모델이 구체적인 과거 기억이나 소스코드가 필요하다고 판단하면, 도구(`read_file`, `session_recall`, `grep_search`)를 호출하여 해당 세부 맥락만 온디맨드로 인출합니다.

- 무제한 토큰 폭발을 방지하고 필요한 세부 맥락만 동적으로 주입

🔄 작동 종료 후 라이프사이클: 백그라운드 메모리 업데이트 (Post-Execution Lifecycle)

사용자와의 대화 및 태스크 실행 루프가 종료되면, 백그라운드 워커가 동작하여 1) 새로운 취향·결정 팩트 추출 및 의미 기억(USER.md) 갱신, 2) 세션 대화 로그의 FTS5 색인화 및 일화 기억 보존, 3) 다음 세션을 위한 진행 상태(progress.md) 동기화를 비동기로 완수합니다.

프롬프트 관심사 분리

목차형 지시문 분리 설계 및 프롬프트 관리

project_root/

- AGENTS.md (전역 네비게이션 지도)
- docs/ (기능별 상세 규정 모듈)
 - refund_policy.md
 - coding_standards.md
 - db_security_rules.md
- skills/ (도구 교본)
- MEMORY.md (핵심 팩트 요약 인덱스)

💡 AGENTS.md는 최소 고정 토큰으로 파일 트리의 역할과 docs/ 목록만 링크하고, 에이전트가 필요 시에만 해당 파일을 동적 로딩합니다.

1. 관심사 분리 (Separation of Concerns)

모든 업무 규칙과 예외 처리를 단일 거대 프롬프트에 몰아넣는 안티패턴을 배제하고, 지침을 성격별로 독립된 마크다운 문서로 분리 모듈화합니다.

2. AGENTS.md: 전역 네비게이션 트리 맵

에이전트에게 전체 파일시스템의 구조, 작업 방향성, 역할 요약만 제공하여 최소한의 고정 토큰으로 전체 맥락을 지도처럼 쥐여줍니다.

3. docs/ 모듈: 적시 동적 주입 (JIT Loading)

환불 규정, DB 보안 규칙 등 기능 단위 문서를 docs/*.md로 보관하고, 런타임에 에이전트가 필요한 문건만 도구로 골라 읽도록 설계하여 토큰 낭비를 원천 차단합니다.

프롬프트 조립 구현

프롬프트 계층화 설계 및 동적 주입 패턴

범용 3계층 프롬프트 조립 개념도

L1. System Core (고정 가이드라인 ➔ 100% 캐시 고정)

- 에이전트 정체성 & 안전성 헌법 수칙
- 알파벳 이름순으로 정렬된 도구 스펙 선언

+

L2. Dynamic Context (동적 환경 정보)

- 현재 작업 디렉토리(CWD) 및 프로젝트 상태
- docs/ 에서 선별 주입된 규칙 및 MEMORY.md 인덱스

+

L3. Task Instruction (최신 임무 지침)

- 현재 턴에서 해결해야 할 사용자 질의 및 명령어

Layer 1: System Core (정적 핵심 수칙)

에이전트의 역할 페르소나, 안전 수칙, 그리고 이름순으로 정렬된 도구 명세처럼 변하지 않는 최상단 고정 규칙을 주입하여 캐시 히트율(KV-Cache Hit)을 극대화합니다.

Layer 2: Dynamic Context (동적 맥락 정보)

CWD 경로, 현재 날짜/시간, 그리고 작업 도중 에이전트 요청에 따라 docs/에서 부분 수집된 비즈니스 규칙과 MEMORY.md 인덱스 등 가변적인 환경 정보를 결합합니다.

Layer 3: Task Instruction (즉각적 지시문)

사용자가 방금 요청한 프롬프트나 에이전트가 이번 루프 턴에서 즉각 실행해야 할 구체적인 Task 지침입니다. 어텐션(Lost in the middle 회피)을 높이기 위해 최하단에 주입합니다.

사례 연구 (CLAUDE CODE)

5계층 프롬프트 조립 구조와 DYNAMIC_BOUNDARY 가드레일

💡 범용 3계층 모델을 Claude Code는 캐시 효율(Prompt Caching)과 메모리 조립을 위해 **완전한 5계층(L1~L5)**으로 세분화합니다.

L1 System Identity & Core Role (정적 기본 지침)

100% 캐시 고정

L2 Capabilities: Tools & Skills (알파벳 정렬 도구)

Schema 캐시 고정

⚡ **__SYSTEM_PROMPT_DYNAMIC_BOUNDARY__** (정적 캐시 한계 가름선)**L3 Dynamic Session Context** (동적 세션 환경)

CWD, Session ID, OS

L4 Semantic Memory & Docs (장기 기억 스냅샷)

MEMORY.md, USER.md 전문

L5 User Context & Task (사용자 지시 및 맥락)

AGENT.md, Git Diff, 현재 턴

🔒 Prefix 캐싱과 Alphabetical Sort

Anthropic의 Ephemeral Cache는 앞단이 단 1글자만 바뀌어도 캐시가 무효화 (Cache Miss)됩니다. L1 지침 뒤의 L2 도구 스펙 배열을 **알파벳 순서로 강제 정렬**하여 도구 선언 순서 변동에 따른 불필요한 캐시 깨짐을 원천 방지합니다.

⚡ Dynamic Boundary를 통한 90% 비용 절감

전체 프롬프트의 70~80%에 달하는 L1, L2(약 12,000~15,000 토큰)를 가름선 위단에 락업(Lock-up)하여 고정하고, L3(세션 환경), L4(메모리), L5(태스크)는 아랫단에 배치하여 캐시 히트된 입력 토큰 단가를 **90% 할인**받습니다.

에이전트 RAG

에이전트 RAG 스택과 하이브리드 라우팅

 Semantic Search

[Vector DB]

- **원리:** 임베딩 벡터 간 코사인 유사도를 측정하여 검색.
- **강점:** 어휘가 달라도 문맥적 의도가 통하는 지식 발췌.
- **사례:** 비즈니스 가이드라인, 정책 집, 고객 문의 FAQ.

 GraphRAG

[Graph DB]

- **원리:** 엔티티 간 관계망을 다단계 경로(Multi-Hop) 추적.
- **강점:** 전역적(Global) 주제 요약 및 아키텍처 의존성 파악.
- **사례:** 소스코드 모듈 의존성, 인물/사건 관계 분석.

 Domain Wiki

[LLMwiki / Docs]

- **원리:** 사내 고유 약어, 규정을 확정(Deterministic) 매칭.
- **강점:** 벡터 검색의 모호성을 배제하고 100% 일치 보장.
- **사례:** 보안 검증 룰, 전사 표준 용어집, 하드웨어 사양 맵.

 Text2SQL

[Structured DB]

- **원리:** 자연어 질의를 SQL 쿼리로 변환하여 RDBMS 실시간 조회.
- **강점:** RAG 요약 환각 없이 수치 통계와 실시간 집계 완벽 보장.
- **사례:** 일일 매출 집계, 주문 이력 통계, 사용자 상태 조회.

💡 에이전트 RAG 라우팅 설계 원칙: 질문의 모호성(비정형 ➡ Semantic), 구조 복잡도(다단계 ➡ Graph), 절대 규칙(확정 ➡ Wiki), 정확한 수치(통계 ➡ Text2SQL)에 따라 하이브리드 라우팅합니다.

장기 메모리 이론

인공지능 모델 기반 장기 메모리 아키텍처

인간 인지 모델(ACT-R/SOAR)과 에이전트 메모리

 작업 기억 (Working Memory)

현재 턴의 대화 맥락, 활성 상태, 단기 추론 버퍼 (RAM)

▲ 인출(Retrieval)

▼ 영속화(Consolidation)

 장기 기억 (Long-Term Memory)

절차적 기억
(도구/규칙)

의미적 기억
(지식/팩트)

일화적 기억
(과거 세션)

1. SOAR / ACT-R 인지 모델 계승

인간의 두뇌가 문제 해결 시 유한한 작업 기억(단기)과 장기 기억을 끊임없이 상호 교환하며 추론하는 메커니즘을 에이전트 시스템에 투영합니다.

2. 단기 작업 기억과 3대 장기 기억

사용자 입력은 먼저 단기 작업 기억에 인입되며, 복잡한 태스크를 해결하기 위해 절차적(도구 명세), 의미적(팩트), 일화적(과거 경험) 장기 지식을 조회·결합합니다.

3. 실행과 피드백의 순환 루프

작업 기억에서 도출된 결정은 도구 실행(Tool Action)으로 이어지고, 환경에서 반환된 관측 결과(Observation)는 다시 작업 기억에 반영되어 메모리 영속화로 순환합니다.

장기 메모리 구현

파일시스템 기반 메모리 아키텍처

📁 로컬 파일시스템 기반 장기 기억 매핑

⚙️ 절차적 기억 (Procedural Memory)

AGENTS.md, mcp.json, skills/*.py

에이전트가 어떤 순서로 작업하고 어떤 도구를 다루는지 규정

📖 의미적 기억 (Semantic Memory)

MEMORY.md, USER.md, knowledge.md

사용자 프로필, 프로젝트 규칙, 도메인 팩트 요약 노트

📅 일화적 기억 (Episodic Memory)

state.db (SQLite FTS5), conversations/*.json

과거 세션별 대화 원문, 성공/실패 궤적 로그 및 세션 색인

1. 절차적 기억 (Procedural Memory)

에이전트가 어떤 규칙으로 작업하고 어떤 도구를 사용하는지 규정하는 지침입니다. 로컬 파일시스템 상의 AGENTS.md와 mcp.json에 안전하게 영속 보존됩니다.

2. 의미적 기억 (Semantic Memory)

사용자 선호도, 도메인 사실 관계 등 축적된 지식입니다. 각 에이전트 작업 공간 내부의 USER.md 및 MEMORY.md 마크다운 파일로 관리됩니다.

3. 일화적 기억 (Episodic Memory)

과거 대화 세션의 상세 내역과 문제 해결 궤적입니다. 세션별 대화 로그 (conversations/)와 SQLite FTS5 데이터베이스(state.db)에 영속화됩니다.

장기 메모리 분류 체계

4계층 영속 메모리 아키텍처 (L1~L4)

L1. Working Memory (단기 작업 기억)

인메모리 대화 메시지 버퍼 (RAM) ➔ 세션 종료 시 초기화

L2. Episodic Memory (일화적 기억)

SQLite DB (state.db) 및 세션 로그 ➔ 턴 종료 시 비동기 저장 & FTS5 색인

L3. Semantic Memory (의미적 기억)

로컬 마크다운 파일 (MEMORY.md / USER.md) ➔ 팩트 추출 후 병합(Consolidation)

L4. Procedural Memory (절차적 기억)

로컬 파이썬 스킬 코드 및 카탈로그 (skills/*.py) ➔ 에이전트 스킬 학습 및 누적



1단계: 메모리 인덱스 자동 주입

기억 목록의 간략한 요약 지도인 MEMORY.md 인덱스는 별도의 도구 호출 없이 매 턴 시스템 프롬프트에 상시 자동 주입되어 AI의 기본 인식을 형성합니다.



2단계: 기억 본문 온디맨드 로딩

인덱스를 읽은 모델이 과거 특정 대화나 상세 지침이 필요하다고 판단하면, 스스로 `session_recall`이나 `read_file` 도구를 호출해 상세 내역을 동적으로 로딩합니다.

기억의 파생과 유기적 결합

L1 작업 기억에서 L2·L3·L4 장기 기억으로의 전이 흐름

L1: Working Memory — 실시간 인메모리 버퍼
messages[] 대화 리스트 + 실행 상태 (RAM)

↓ 1. 세션 종료 시 비동기 flush

↓ 2. 백그라운드 리뷰 팩트 추출·병합

↓ 3. 스킬 생성 도구 호출

↑ session_recall 도구로 온디맨드 인출

↑ 시스템 프롬프트에 스냅샷 자동 주입

↑ 도구 카탈로그에 스펙 상시 주입

L2: Episodic Memory — 과거 세션 데이터
SessionDB (state.db) + FTS5 세션 인덱스

L3: Semantic Memory — 요약된 지식과 성향
MemoryStore ➔ MEMORY.md / USER.md

L4: Procedural Memory — 스킬 교본
~/.hermes/skills/*.py (실행 코드)

📁 실시간 ➔ 장기 영속화 (WRITE)

작업 버퍼(L1)에서 생성된 전체 기록은 **SQLite(L2)**에 비동기 저장되고, 백그라운드 분석을 거쳐 사용자 전역 규칙/팩트는 **L3(MEMORY.md)**로, 새로 학습된 파이썬 스킬 코드는 **L4(skills/)**로 분류 저장됩니다.

📖 장기 영속 지식 ➔ 런타임 인출 (READ)

L3의 핵심 기억과 L4 스킬 인덱스는 **시스템 프롬프트에 자동 주입**되어 AI가 상시 인지하며, 오랜 과거 대화 및 세부 사건 데이터(L2)는 AI가 필요한 시점에만 session_recall 도구 호출로 온디맨드 인출합니다.

파일시스템 기반 영속화

파일시스템 및 로컬 DB 기반 장기 메모리 저장 구조

~/.agent_workspace/

- **skills/** (L4 절차 기억: 실행 스크립트)
 - search_github.py
 - test_runner.py
- **conversations/** (L2 일화 기억 로그)
 - 2026-09-01_task1.json
 - state.db (FTS5 가상 테이블)
- **MEMORY.md** (L3 의미 기억: 프로젝트 지식)
- **USER.md** (L3 의미 기억: 사용자 선호도)

💡 복잡한 외부 벡터 DB 없이도 **마크다운 파일**과 **SQLite FTS5**만으로 완벽한 영속 메모리 하네스를 구축할 수 있습니다.

1. 모듈식 서브 에이전트 구성 (L4 절차적 기억)

각각의 서브 에이전트가 자체적인 AGENTS.md와 skills/*.py 실행 코드를 독립적으로 보존하여 절차적 기억을 캡슐화합니다.

2. 마크다운 기반 자기 갱신 (L3 의미적 기억)

에이전트가 학습한 사용자 성향이나 프로젝트 규칙을 불투명한 DB 대신 USER.md, MEMORY.md에 텍스트로 직접 갱신·누적합니다.

3. 세션 로그 및 FTS5 색인화 (L2 일화적 기억)

과거 세션의 대화 내역은 conversations/*.json 로그와 SQLite state.db에 안전하게 보존되며, 전문 검색(FTS5)을 위해 색인화되어 에이전트가 언제든지 과거 경험을 온디맨드로 인출할 수 있습니다.

메모리 쓰기 아키텍처

메모리 저장 시점: 실시간 쓰기(Hot Path) vs 비동기 백그라운드

1. 실시간 런타임 쓰기 (In the hot path)

대화 루프 도중 메모리 분석과 저장을 동기식으로 실행

- 매 턴 사용자 답변을 내보내기 전에 메모리 추출 및 저장을 먼저 완료해야 합니다.
- LLM 팩트 분석 및 DB 기록으로 인해 **1,000~3,000ms 이상의 응답 지연 (Latency)**이 매 턴 누적됩니다.
- 사용자가 대화 흐름이 무겁고 느리다고 체감하여 실무 만족도가 급격히 저하됩니다.

2. 비동기 백그라운드 쓰기 (In the background) ★ 추천

대화 루프(L1)와 기억 영속화(L2~L4)를 완전히 분리

- 사용자에게는 추론 즉시 스트리밍으로 답변을 먼저 반환합니다 (지연 시간 0ms).
- 세션 종료 또는 턴 완료 후 **백그라운드 데몬 스레드**(`daemon=True`)가 가동되어 메모리를 추출하고 DB에 갱신합니다.
- 엔터프라이즈 프로덕션 환경의 **표준 메모리 라이프사이클 정책**입니다.

💡 프로덕션 규칙: 사용자 대화 경험을 지키기 위해 메모리 저장 파이프라인(L1 대화 ➡ L2 로그 저장 ➡ L3 팩트 갱신)은 반드시 비동기 백그라운드 워커로 격리해야 합니다.

장기 메모리 파이프라인 ① 의미 기억 생성

시맨틱 메모리 추출 및 일관성 관리 파이프라인

01

Fact Extraction (팩트 추출)

대화가 종료되면 LLM이 최근 메시지를 분석하여 사용자의 선호도, 기술 결정, 프로젝트 팩트를 원자적 문장 단위로 추출합니다.

예: "User prefers async/await over callbacks"



02

Semantic Retrieval (기존 기억 비교)

추출된 팩트의 키워드를 기반으로 기존 USER.md, MEMORY.md에 이미 존재하는 엔트리를 검색하여, 모순되거나 갱신이 필요한 기존 지식을 로드합니다.



03

Consolidation Decision (4대 정제 매트릭스)

새 팩트와 기존 기억을 비교 분석한 후, 4대 결정 매트릭스에 매핑하여 최종 갱신합니다:

- **ADD**: 새로운 팩트 신규 등록
- **UPDATE**: 기존 데이터 보완 수정 (replace)
- **DELETE**: 만료되거나 모순된 정보 삭제
- **NOOP**: 이미 존재하는 동일 사실 무시

 **USER.md** 섹션 엔트리 갱신 예시:

- [기존] § Hobbies: 주말마다 등산과 테니스를 즐김
+ [UPDATE 후] § Hobbies: 최근 테니스 부상으로 실내 수영으로 전환함

장기 메모리 파이프라인 ② 일화 기억 생성

세션 데이터 기반 일화 기억 생성: 2단계 요약 및 키워드 추출

1단계 로컬 규칙 기반 원문 키워드 추출

대화 원문에 사용자가 직접 발화한 한글/영문 단어를 형태소 및 불용어 필터링으로 1차 추출합니다.

- 원문 단어 100% 손실 방지: "쿠키", "보안", "JWT", "도커" 등 실제 언급 단어 보존
- LLM 번역으로 인한 한글 고유명사 유실 방지

2단계 LLM 지능형 요약 및 개념어 보완

LLM이 세션 전체 맥락을 읽고 핵심 결정 1줄 요약문 (Summary)을 도출하고 원문에 없는 개념어를 보충합니다.

- 상위 개념 및 영문 매핑: "Stateless Authentication", "XSS Mitigation", "Token Security" 등 추출
- 대화 전반의 아키텍처 의사결정 맥락 압축

3단계 SQLite FTS5 하이브리드 색인화

세션 ID + 요약문 + (원문 키워드 U LLM 보완 키워드)를 SQLite FTS5 가상 테이블에 저장합니다.

- 한국어 원문("쿠키 보안") 검색 성공률 100%
- 영문 기술어("Stateless Auth") 검색 성공률 100%

💡 **핵심 설계 원칙 (원문 단어 손실 0%):** LLM에게만 요약과 키워드를 맡기면 사용자가 실제 말한 한글 표현을 영어로 번역해버려 한국어 검색 시 누락이 발생합니다. 로컬 규칙 추출(1단계)과 LLM 지능형 보완(2단계)을 결합하여 검색 누락을 원천 차단합니다.

장기 메모리 파이프라인 ② 일화 기억 인출

과거 세션 대화 복원: 키워드 검색 기반 2단계 일화 기억 인출

2-Stage JIT 회상 (Just-In-Time Recall)

1단계. `search_sessions('검색어')` ➔ SQLite FTS5 세션 탐색
가벼운 세션 요약문(Summary) 및 키워드 식별

▼ (상세 대화 복원 필요 판단 시)

2단계. `session_recall(session_id, keyword)` 도구 호출
핵심 Anchor 단어 매칭 턴 핀포인트 타겟팅

▼ (전후 맥락 윈도우 슬라이싱)

3단계. Anchored View 생성 (Anchor $\pm$ 2턴 + 복엔드)
과거 세션 시작/끝 메시지 + Anchor 전후 맥락만 슬라이싱



4단계. 현재 대화 턴의 작업 기억(L1)으로 온디맨드 주입

⚙ 왜 세션 전체가 아닌 2단계 인출인가?

새 세션이 시작되면 이전 세션의 L1 작업 기억은 비어 있습니다. 과거 세션의 수십 개 대화를 통째로 프롬프트에 부으면 즉시 컨텍스트가 폭발하므로, 1단계에서 세션 요약문만 가볍게 탐색한 후, 모델이 세부 내역을 요구할 때만 2단계에서 정밀 대화만 슬라이싱합니다.

🛡 Anchor-Lineage 기반 맥락 보존

키워드가 매칭된 단일 메시지만 가져오면 앞뒤 사정을 몰라 인지 왜곡(Hallucination)이 생깁니다. Anchor 매칭 턴 주변 ± 2 턴 윈도우와 해당 세션의 시작/끝 복엔드 메시지를 세트로 인출하여 대화의 인과관계를 온전히 복원합니다.

장기 메모리 종합 아키텍처

통합 장기 메모리 아키텍처: 다중 계층 저장소와 런타임 제어

통합 4계층 장기 메모리 제어 시스템

🧠 런타임 제어기 & 프롬프트 조립기 (Prompt Assembler)

사용자 요청 인입 → 도구/RAG/메모리 스냅샷 병합 → LLM 추론

▲ READ (동기 주입) | ▼ WRITE (비동기 영속화)

📁 목적별 다중 계층 로컬 저장소

L3 사용자 선호 (USER.md)

성향, 톤앤매너, 코딩 스타일

L2 세션 데이터 (state.db)

대화 로그, FTS5 색인, Anchor

L4 에이전트 스킬 (skills/)

자율 학습된 파이썬 스크립트

L3 프로젝트 지식 (MEMORY.md)

도메인 규정, 아키텍처 팩트

⚡ 비동기 백그라운드 워커 (daemon=True)

팩트 정제 매트릭스(ADD/UPDATE) → FTS5 색인화 → 세션 요약 갱신

1. 입력 및 런타임 조립 (Prompt Assembler)

사용자 요청을 입력받아 L3 핵심 스냅샷과 L4 스킬 목록을 사전 주입하고, 필요 시 도구를 통해 L2 세션 대화를 온디맨드 결합하여 LLM에 송신합니다.

2. 다중 계층 로컬 저장소 (Multi-Tier Storage)

메모리를 사용자 선호(USER.md), 대화 이력(state.db), 에이전트 기술(skills/), 프로젝트 지식(MEMORY.md)으로 목적별 격리하여 컨텍스트 오염을 원천 차단합니다.

3. 비동기 백그라운드 루프 (0ms Latency)

사용자 대화 응답 지연을 방지하기 위해 실시간 추론 경로에서 격리 구동되며, 팩트 병합, 일화 세션 색인화, 오래된 기억의 정리를 백그라운드에서 완수합니다.

세션 인계

장시간 실행 에이전트의 세션 인계(Handoff) 설계

progress.md 기반 상태 보존 및 인수인계

세션 A (컨텍스트 한계 도달 직전)

수천 줄의 대화 누적으로 토큰 한계 접근 → 세션 분할 결정

▼ [세션 종료 전 자율 덤프]

📄 작업 현황판 (progress.md)

- 완료된 마일스톤 (Done Tasks)
- 남은 작업 목록 (TODO Items)
- 핵심 파일 변경 내역 및 환경 변수 스냅샷

▼ [새 세션 시작 시 즉시 로딩]

세션 B (가벼운 재진입 / Clean Context)

과거 대화 전체 전송 없이 progress.md만 읽고 즉각 작업 재개!

1. 세션 인계(Handoff)의 필요성

수 시간 이상 실행되는 대형 태스크는 단일 세션에서 계속 구동 시 누적 로그로 인해 컨텍스트가 임계치를 초과하고 어텐션이 마비되어 오작동합니다.

2. 작업 현황판 (progress.md) 중심 설계

세션 종료 전, 에이전트가 스스로 완료한 마일스톤, 남은 작업(TODO), 변경점 등을 progress.md에 구조화 정보로 기록하고 세션을 안전하게 닫습니다.

3. 가볍고 안전한 세션 재진입 (Clean Re-entry)

새 세션이 기동할 때 수만 라인의 이전 대화를 다시 전송하지 않고, 파일시스템에 적재된 progress.md 지표만 읽어 즉각 이전 맥락과 동기화합니다.

컨텍스트 최적화

컨텍스트 창 크기 제어: 5단계 컴팩션 방어선과 Amnesia Guard

컴팩션 방어선	주요 대상 (Target)	구현 메커니즘 (Mechanism)	토큰 절감률
1. Snip Compactor (도구 결과 축약)	2턴 이전의 오래된 도구 실행 결과	에이전트가 이미 읽고 판단을 끝낸 과거 도구 출력을 [Tool result snipped: 500 chars] 1줄 스텝으로 즉시 압축.	~90%
2. Micro Compactor (대형 출력 디스크 스왑)	최근 턴의 5,000자 초과 대형 웹/문서 결과	대형 출력을 디스크(/swaps/swap_*.txt)로 격리하고, 에이전트에게 부분 인출할 수 있는 Actionable Hint 제공.	~99%
3. Context Collapse (탐색 시퀀스 접기)	연속된 순수 조회 도구 (list_dir, grep, read)	수많은 자잘한 탐색 도구들을 [Context Collapsed: N steps]로 접어 Auto-Compact를 사전 방어 (write_file 등 변경 도구는 100% 보존).	~95%
4. Auto Compactor & Amnesia Guard	8,000 토큰 초과 누적 대화 전체	전체 대화를 4개 섹션(목표, 결정, 변경점, 다음 작업)으로 요약한 후, 최근 작업 파일의 Head(40줄) + Tail(10줄) 및 활성 계획 자동 재주입.	~98%
5. Reactive Compactor (비상 탈출 복구)	API 호출 시 413 Payload Too Large 에러 발생	인덱스가 아닌 HumanMessage 턴 경계 단위 20% Silent Slicing을 수행하여 고아 ToolMessage 에러 없이 자동 재시도 복구.	긴급 회복

🛡️ 기억 상실(Amnesia) 방지 원리: 토큰을 아끼려고 대화를 단순히 요약문으로 뭉개면 방금 고친 코드나 세부 지침을 잊어버립니다. Amnesia Guard가 핵심 파일의 머리/꼬리와 진행 계획을 SystemMessage로 스마트 재주입하여 연속성을 유지합니다.

요약 및 정리

3교시 핵심 요약 및 정리

컨텍스트 통제와 프롬프트 5계층 스택

- LLM은 Stateless 엔진일 뿐이며 유한한 컨텍스트 창을 RAM처럼 조율하는 주체는 실행 하네스입니다. 시스템 코어(L1)와 정렬된 도구(L2)를 Boundary Marker 윗단에 락업하여 캐시 히트 단가를 90% 절감하고, 세션 환경(L3), 메모리(L4), 사용자 지시(L5)를 계층화 조립합니다.

하이브리드 RAG 스택 구성

- 의미론적 비정형 검색(Semantic Search), 다단계 관계망 추적(GraphRAG), 고정 규정 일대일 매칭(Domain Wiki), 그리고 정밀 집계를 위한 수치 계산(Text2SQL)을 에이전트 라우팅 목적에 맞춰 적재적소에 결합 및 설계해야 합니다.

4계층 영속 메모리 라이프사이클 (L1~L4)

- RAM 작업 기억(L1)에서 파생된 기록들은 비동기 백그라운드 워커를 거쳐 SQLite 세션 로그(L2), 마크다운 의미 기억(L3), 파이썬 스킵 코드(L4)로 영속화됩니다. L3는 4대 정제 매트릭스(ADD/UPDATE)로 일관성을 유지하며, L2는 1단계 원문 키워드 보존과 2단계 LLM 개념어 보완을 거쳐 색인됩니다.

2-Stage JIT 일화 기억 회상과 5단계 컴팩션 방어선

- 과거 세션 대화는 가벼운 요약 탐색 후 필요한 시점에만 Anchor 윈도우 슬라이싱으로 온디맨드 인출합니다. 또한 대화가 길어질 때는 Snip ➡ Micro ➡ Collapse ➡ Auto+Amnesia ➡ Reactive로 이어지는 5단계 방어선 미들웨어로 기억 상실 없이 컨텍스트를 방어합니다.

MODULE 04

도구 엔지니어링 및 커스텀 MCP 구축

에이전트-컴퓨터 인터페이스(ACI) 최적화 설계부터 JIT 즉석 코딩 패러다임 비교, 그리고 Model Context Protocol(MCP) 표준 구축을 다룹니다.

🎯 4교시 핵심 학습 로드맵

1. 에이전트 친화적 ACI 설계

Docstring 컨텍스트 최적화 및 자가 치유를 위한 실행 가능한 에러 힌트(Semantic Hint) 구현

2. 도구 폭발 억제 & 스킬 순환

인지 과부하를 막는 JIT 점진적 도구 노출과 SKILL.md 기반의 자율 지식 영속화 루프

3. MCP 표준 프로토콜 & FastMCP

N×M 결합도를 깨뜨리는 MCP 표준 아키텍처와 데코레이터 기반 초고속 도구 서버 구축

도구 설계 방법론

좋은 도구를 설계하는 법: ACI 스키마와 에러 피드백

```

from pydantic import BaseModel, Field
from langchain_core.tools import tool

# 1. Pydantic 기반 ACI 스키마 정의 (LLM의 도구 선택 컨텍스트)
class FileReadInput(BaseModel):
    file_path: str = Field(description="읽을 파일의 상대/절대 경로")
    offset: int = Field(default=1, description="시작 줄 번호 (1-indexed)")
    limit: int = Field(default=200, description="최대 조회 줄 수 (페이징)")

# 2. @tool 등록 및 명확한 의도의 Docstring
@tool(args_schema=FileReadInput)
def file_read(file_path: str, offset: int = 1, limit: int = 200) -> str:
    """파일 내용을 페이징하여 정돈된 라인 단위로 안전하게 조회합니다."""
    abs_path = os.path.abspath(file_path)
    if not os.path.exists(abs_path):
        # 3. 단순 Traceback 대신 실행 가능한 복구 힌트(Hint) 제공
        similar = find_similar_files(file_path)[:3]
        return f"FileRead Error: '{file_path}' 파일 없음. 유사 제안: {similar}"

    # 4. 컴팩트한 ACI 규격 출력 (메타데이터 헤더 + 줄 번호)
    lines = read_lines(abs_path, offset, limit)
    return f"[File: {file_path} (Lines {offset}-{offset+len(lines)})]\n" +
lines

```

1. 도구 명세(Specification)는 곧 컨텍스트

도구 이름, Docstring, Pydantic Field(description=...)는 LLM이 도구 선택 여부와 인자 형식을 판단하는 핵심 컨텍스트입니다. 인자 제약조건과 기본값을 명확히 정의해야 인자 누락 환각을 방지합니다.

2. 에이전트-컴퓨터 인터페이스 (ACI)

인간용 마크업 대신, 에이전트가 즉각 파싱하고 컨텍스트 윈도우를 아낄 수 있는 컴팩트한 헤더 메타데이터와 라인 번호 형식으로 반환하여 환각(Hallucination)을 원천 차단합니다.

3. 실행 가능한 에러 피드백 (Actionable Error Hint)

도구 실행 실패 시 단순 시스템 Traceback만 반환하면 에이전트가 오류를 교정하지 못하고 헤맬니다. "유사 파일명 제안", "올바른 줄 번호 범위 안내" 등 실행 가능한 힌트를 제공해야 스스로 자가 치유(Self-Correction)할 수 있습니다.

도구 호출 패러다임

Tool Calling (JSON) vs CodeAct (Python 실행) 비교

비교 항목	정적 Tool Calling (JSON 기반 호출)	CodeAct 방식 (Python 런타임 JIT 실행)
작업 흐름 제어 (Workflow Control)	단발성에 최적. 병렬 도구 호출(Parallel Call) 로 독립 작업은 1턴에 동시 처리 가능하나, 순차 의존(B가 A 결과를 필요로 할 때) 작업은 턴 수 폭증.	복합 로직 단일 턴 완수. for 루프, if 조건 분기 및 순차 의존 연산을 파이썬 코드 내부에서 일괄 해결.
토큰 및 비용 구조 (Cost & Tokens)	출력 토큰 최소화 및 경량 모델(Haiku/Mini) 활용 가능. 단, 도구 수가 많아지면 도구 스키마(JSON Schema) 입력 토큰이 매 턴 누적 청구됨 (캐싱 필수).	코드 작성에 따른 출력 토큰 증가 및 고성능 코딩 모델 요구. 단, 대량 데이터 집계 시 원시 데이터의 대화 컨텍스트 오염을 원천 차단 .
태스크 성공률 (Benchmark Accuracy)	기준선 (다단계 순차 작업 시 턴 누적으로 인한 중간 탈락 및 인자 환각 위험 증가).	복합 작업(M3ToolEval) 기준 약 20%p 향상 . try-except 기반 실행 런타임 피드백 자가 디버깅(Self-debugging) 이 핵심 요인.
실행 지연시간 (Latency & Speed)	매우 빠름 (밀리초 단위) . 호스트 네이티브 API를 즉각 호출하므로 실시간 사용자 인터랙션 응답성 우수 (순차 멀티턴 시에는 누적 지연 발생).	상대적 지연 (초 단위) . 파이썬 프로세스 기동, 인터프리터 런타임 해석, 표준출력 캡처 등 환경 오버헤드 상존 (단, 10턴 이상 복합 태스크에서는 총 지연 역전 가능).
유연성 및 상태 관리 (Flexibility & State)	제한적 (Deterministic) . 사전 정의된 스킴 외 행동 불가. 중간 계산 상태를 대화 컨텍스트에 텍스트로 보존해야 하므로 컨텍스트 소진.	극대화 (Open-ended) . 파이썬 생태계(pandas, math 등) 100% 활용. 중간 연산 변수를 로컬/프로세스 메모리에 직접 유지 하여 컨텍스트 무부담.
안전성 및 인프라 (Security & Infra)	매우 높음 (Pydantic 스킴 사전 유효성 검증 및 화이트리스트 기반 제어가 쉬워 호스트 시스템에 무영향).	격리 인프라 권장 (임의 코드 실행 및 무한 루프 위험 방지를 위해 프로덕션 배포 시 격리 인프라 권장 / 실습·개발 환경은 가상환경 및 타임아웃 제어로 안전하게 운영 가능).

💡 실무 아키텍처 결론: 일반적인 조회·정형화된 API 호출은 정적 Tool Calling으로 빠르고 저렴하게 처리하고, 대량 데이터 가공·반복 연산·동적 예외 해결은 CodeAct 코드 실행(JIT)에 위임하는 하이브리드(Hybrid) 설계가 최적입니다.

기본 도구셋

에이전트 3대 기본 도구셋 (Base Toolset) 구성도

 웹 검색 및 수집

외부 인터넷 정보에 접근하여 실시간 데이터를 수집하고 참조 문서를 취합하는 관문 역할을 담당합니다.

- web_search:** 질의 대응 관련 링크 검색
- fetch_webpage:** 특정 URL HTML 수집/파싱
- 실시간 최신 동향 조사 및 레퍼런스 확보

 로컬 파일 시스템

에이전트가 작업 공간(Workspace) 내에서 코드를 수정하고, 진행 경과 및 산출물을 영속 기록하는 저장소입니다.

- read/write_file:** 파일 조회 및 수정 기록
- grep_search:** 코드 패턴 디렉토리 검색
- 소스코드 탐색, 수정 및 아티팩트 보존

 코드 실행 샌드박스

에이전트가 생성한 소스코드를 격리된 환경에서 안전하게 인터프리팅하고 실행하여 적합성을 검증하는 인프라입니다.

- run_python:** 파이썬 코드 실행/테스트 검증
- run_command:** 터미널 빌드/린트 명령 수행
- 동적 데이터 처리 및 런타임 결과 피드백

 외부 검색 데이터 수집 →  워크스페이스에 파일 적재 & 구조화 →  샌드박스 런타임에서 연산 및 빌드 검증 →  최종 산출 결과물 파일 갱신

도구 폭발 억제

도구 폭발 예방을 위한 점진적 도구 노출 설계

도구 폭발 (Tool Explosion)



⚠ 인지 과부하 & 도구 오호출 급증

점진적 노출 (Progressive)



✓ 어텐션 집중 & 추론 성공률 극대화

1. 도구 과밀(Saturation)에 따른 인지 부하 예방

수십 개 이상의 도구 스키마를 한 번에 주입하면 어텐션이 분산되어 필수 인자를 누락하거나 유사한 도구를 혼동(Tool Confusion)하는 환각이 급증합니다.

2. JIT (Just-In-Time) 점진적 도구 노출

작업 단계(Phase)를 분석하여, 오직 지금 당장 필요한 2~3개의 핵심 도구만 컨텍스트에 동적으로 주입하여 추론 성공률과 토큰 효율을 동시에 보장합니다.

3. 도구 라우팅 미들웨어

사용자 의도를 임베딩 대조하여 도구를 선별 적재하거나, 에이전트가 필요 시 도구 카탈로그를 직접 검색하여 적재하는 지연 로딩(Tool Search) 방식을 결합합니다.

자율 지식 축적

에이전트 자율 스킬(Skills) 순환 구조와 스토리지 구성

자율 스킬 순환 루프 (Closed Loop Architecture)

- 1 **JIT Code Gen:** 미정의 작업 발생 시 LLM이 실행 파이썬 스크립트 자율 생성
- 2 **Sandbox Run:** 격리 터미널에서 코드 실행 및 테스트 결과 정합성 검증
- 3 **Skills Save:** 검증 완료된 코드를 SKILL.md 표준 규격으로 파일 영속화
- 4 **Active Recall:** 후속 세션에서 유사 태스크 인지 시 저장된 스킬 즉각 재사용

💡 반복되는 코딩 작업을 예셋으로 누적하여 점진적으로 성장하는 에이전트 구현

```
workspace_root/  
├── skills/ # 자율 지식 및 스킬 저장소  
│   ├── SKILL.md # 자연어 도구 명세 메타데이터 인덱서  
│   └── scripts/excel_helper.py # 실제 실행 파이썬 모듈
```

SKILL.md 표준 포맷 예시 (YAML Frontmatter)

```
---  
name: "excel_pivot_generator"  
description: "엑셀 데이터를 집계하여 피벗 리포트를 자동 생성하는 도구"  
parameters: ["filepath", "group_col", "target_col"]  
---  
# 자연어 사용 지침: scripts/excel_helper.py를 실행하여 결과를 리턴할 것.
```

사전 구비 스킬 vs 자율 진화 스킬

- **Pre-built Skill:** 개발자가 범용 도구(DB, 웹 수집)를 사전에 선언해 둔 고정 스킬.
- **JIT Generated Skill:** 에이전트가 런타임에 즉석 생성·검증 후 스스로 적재한 자율 확장 스킬.

도구 설계 철학

도구 점진적 공개(Progressive Disclosure)의 철학과 성능

컨텍스트 윈도우 내 유효 어텐션 점유율 비교

 도구 무제한 주입 (100+개 전체 노출)

도구 스펙 해석 (85%)

추론 (15%)

- LLM이 방대한 도구 매개변수를 읽고 구별하는 데 어텐션 자원을 전부 소모하여 오호출 및 인자 누락 위험이 급증합니다.

 점진적 도구 노출 (필수 도구 2~3개만 JIT 주입)

도구 (15%)

사용자 질문 분석 및 CoT 논리 추론 집중 (85%)

- 불필요한 메타데이터가 억제되어 모델이 유저 질문 분석과 심층 추론(CoT)에 연산 자원을 집중하여 성공률을 극대화합니다.

 1. 유한한 인지 한계 (Attention Span) 극복

컨텍스트 길이가 늘어나더라도 모델의 실시간 집중도에는 한계가 있습니다. 작업 단계(Phase)별로 엄격히 정제된 도구 카탈로그만 선택 노출하는 미니멀 아키텍처가 핵심입니다.

 2. 토큰 비용 및 대기 시간(Latency) 절감

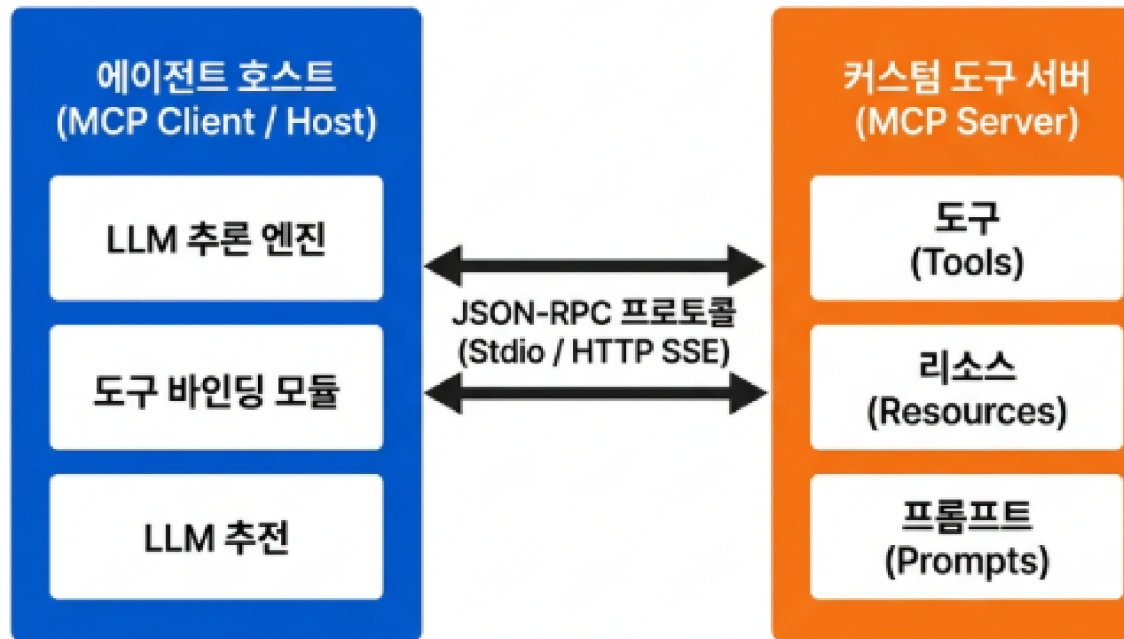
100개 도구 명세를 매 턴마다 반복 주입하면 입력 토큰이 급증해 운영 비용이 폭증합니다. 꼭 필요한 도구만 JIT로 로드하는 것이 프로덕션 운영의 필수 요건입니다.

 3. 자율 도구 검색(JIT Tool Routing) 결합

에이전트가 복합 문제를 해결할 때 스스로 도구 저장소에서 적합한 도구를 검색해 현재 컨텍스트에 동적으로 주입하는 도구 라우팅 미들웨어 설계를 정착시켜야 합니다.

도구 표준 프로토콜

Model Context Protocol (MCP) 표준 프로토콜 소개



💡 MCP 표준 프로토콜이란?

Anthropic이 주도하는 JSON-RPC 2.0 기반의 오픈소스 표준 규약입니다. 호스트 에이전트(Client)와 외부 도구 실행 서버(Server)의 결합을 분리하여 플러그앤플레이(Plug-and-play) 환경을 제공하며, 최신 스펙은 **완전한 무상태성(Stateless Core)**을 채택하여 세션 핀닝 없는 클라우드 수평 확장을 보장합니다.

✨ 주요 아키텍처 설계 효과

도구 실행 런타임을 물리적으로 서버 단에 격리하여 호스트 에이전트 시스템 보안을 강화하고, 한번 작성한 도구를 Cursor, Claude Desktop 등 이중 클라이언트에 별도 코드 수정 없이 즉시 연결하여 재사용합니다.

🔌 프로토콜 레벨의 안정성 & 무상태성

인증(Authentication), 도구 동적 탐색(Discovery), 자체 **완결형(Self-contained)** 요청 규격이 단일 프로토콜로 통합되어 있어, 파편화된 개별 어댑터 개발 부담을 없애고 서버리스(FaaS) 및 마이크로서비스 환경에서 자유롭게 스케일아웃합니다.

도구 표준 프로토콜

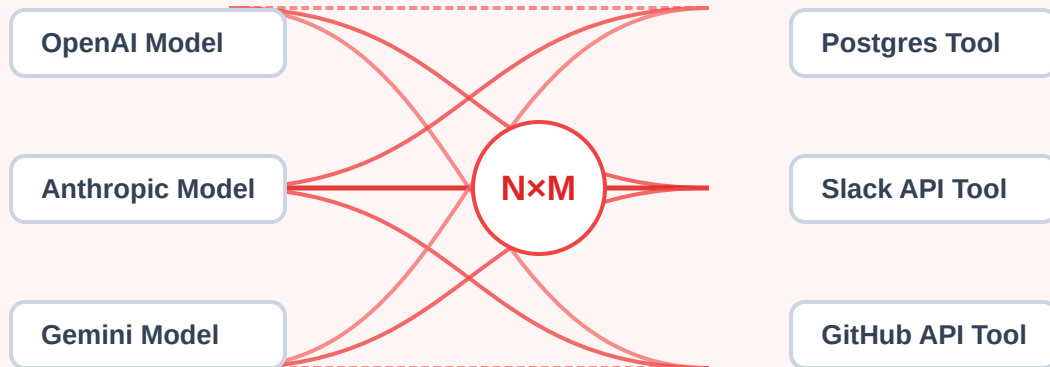
프로토콜 비교: MCP (Model-to-Tool) vs A2A (Agent-to-Agent)

비교 차원	Model Context Protocol (MCP)	Agent-to-Agent Protocol (A2A)
주요 해결 문제	LLM 모델(인프라)과 다양한 외부 도구(API, DB, 파일 등) 간의 인터페이스 표준화	이종의 독립된 에이전트(Agent) 인스턴스들 간의 상태 공유, 협업 및 메시지 위임
통신 주체	Model ↔ Tool Server (JSON-RPC 기반 단방향/스트리밍 호출)	Agent ↔ Agent (비동기 메시지 교환 및 협업 메신저 통신)
상태 관리 모델 (Statefulness)	Stateless (완전 무상태 코어) • 세션 고정(Sticky Session) 없이 모든 요청이 자체 완결(Self-contained). • 다중 턴 상호작용은 컨텍스트 직렬화(Multi Round-Trip)로 처리.	Stateful (장기 상태 보존 및 인계) • 에이전트 간 연속 대화 컨텍스트 및 작업 상태 유지(State Handoff). • 장기 실행 태스크를 위한 비동기 큐 및 감사 경로(Audit Trail) 추적.
적합한 시나리오	에이전트가 로컬 파일에 접근하거나 PostgreSQL DB, AWS 클라우드 API를 안전하게 실행해야 할 때	개발 에이전트가 테스트 에이전트에게 린트 검증을 요청하거나, 상담원 에이전트가 정산 에이전트에게 환불 처리를 이관할 때
주요 기본 요소	정적 명세(Resources), 동적 도구(Tools), 템플릿(Prompts) + Streamable HTTP 단일 엔드포인트	상태 보존(State Handoff), 비동기 큐(Asynchronous Queueing), 감사 추적(Audit Trail)

도구 표준 프로토콜

왜 MCP가 필요한가? (N×M의 복잡도 붕괴 방지)

AS-IS: 기존의 N × M 복잡도 폭발 (Spaghetti Integration)



* 모델이나 도구가 추가될 때마다 N×M개의 전용 어댑터 코드를 매번 중복 개발해야 합니다.

1. 아키텍처 파편화와 중복 개발

제공자(OpenAI, Anthropic 등)마다 도구 바인딩 및 함수 호출 문법이 달라, 동일한 기능의 도구라도 프레임워크별로 수십 개의 개별 연동 코드를 유지보수해야 하는 비효율이 발생합니다.

2. 통제 불가능한 에러 핸들링

예외 처리 규격이 일원화되지 않아, 도구 실패 시 가공되지 않은 시스템 Traceback이 그대로 주입되어 LLM의 컨텍스트를 오염시키고 자율 복구 흐름을 차단합니다.

도구 표준 프로토콜

MCP를 통한 해결 방안 (N + M 아키텍처로의 전환)

TO-BE: MCP 기반 N + M 통합 (Hub-and-Spoke Architecture)



* 단 한 번의 MCP 도구 서버 구축으로 모든 에이전트 호스트에 플러그인 즉시 연동 가능 (N + M)

1. 추론 엔진과 인프라의 완전한 디커플링

MCP 서버는 표준화된 메타데이터 규약으로 도구 사양을 노출하고, 클라이언트는 런타임에 이를 표준 JSON-RPC 호출로 변환하여 모델에 바인딩합니다. 모델 변경 시에도 도구 코드를 재작성할 필요가 없습니다.

2. 일관된 거버넌스 및 자가 치유

인증 자격 증명(Credentials), 권한 통제, 표준 에러 핸들링 규격이 프로토콜 레벨에서 일원화되므로, 중복 코드를 제거하고 에이전트의 자율 교정(Self-Correction) 신뢰성을 극대화합니다.

도구 표준 프로토콜

MCP 서버가 공급하는 3대 기본 요소 (Primitives)

1. Resources (정적 리소스)



- **정의:** LLM 에이전트가 읽기 전용(Read-only)으로 조회할 수 있는 내부 정적 텍스트/바이너리 데이터.
- **사례:** 전자 할인을 규정, 시스템 에러 로그, 로컬 코드 원본 파일.
- **특징:** 특정 URI 규격(audit://policy)으로 에이전트 컨텍스트에 안전하게 바인딩.

```
# URI 스키마 바인딩 예시
uri: "audit://company/policy"
mimeType: "text/plain"
```

2. Prompts (지침 템플릿)



- **정의:** 에이전트 런타임이 사전 정의해두고 동적으로 호출할 수 있는 시스템/사용자 프롬프트 템플릿.
- **사례:** 보안 감사 전문 지침, 코드 리뷰 표준 프롬프트.
- **특징:** 인자값(Arguments) 슬롯을 두어 런타임에 동적으로 최적 프롬프트 조립.

```
# 프롬프트 템플릿 슬롯 예시
name: "code_audit"
arguments: ["target_lang", "strict_mode"]
```

3. Tools (동적 도구)



- **정의:** LLM 에이전트가 호출하여 외부 환경과 상호작용하고 상태를 갱신하는 능동형 함수.
- **사례:** 데이터베이스 SQL 실행기, 셸 명령 터미널, Slack 알림 발송.
- **특징:** 인자 JSON을 받아 샌드박스에서 함수를 실행한 후 표준 결과를 반환.

```
# 실행형 도구 JSON 스키마
name: "execute_query"
parameters: {"sql": {"type": "string"}}
```

도구 표준 프로토콜

MCP 전송 계층 비교: Stdio vs Streamable HTTP

비교 차원	Stdio Transport (로컬 파이프 IPC)	Streamable HTTP Transport (무상태 웹 표준)
1. 통신 및 연결 방식	로컬 프로세스 표준 입출력(IPC) 호스트 에이전트가 도구 서버를 직접 하위 프로세스로 기동하여, Stdin/Stdout 파이프를 OS 내부에서 지연 없이 직접 통신.	단일 엔드포인트 HTTP 통신 표준 HTTP POST 단일 엔드포인트로 JSON-RPC를 전달하고 단일 채널에서 스트리밍(SSE) 또는 일괄 응답 반환 (기존 분리형 듀얼 SSE 대체).
2. 세션 및 상태 관리 (Statefulness)	프로세스 수명주기 종속 서브프로세스가 실행 중인 동안에만 메모리 상태가 유지되며, 프로세스 종료 시 완전히 소멸.	완전한 무상태성 (Stateless Core) • 기존: 서버가 세션 상태를 유지해야 하므로 특정 인스턴스에 종속(세션 고정/Sticky Session 병목). • 최신: 각 요청이 인증·기능 메타데이터(_meta/헤더)를 독립 포함(자체 완결)하여, 어떤 서버 인스턴스도 즉시 독립 처리 가능.
3. 물리 배포 환경	로컬 단일 환경 (단일 사용자) 개인 개발 환경 및 단일 시스템 기반 CLI 에이전트(Claude Code, Cursor 등) 탑재에 최적화.	원격 분산 및 서버리스 (Cloud-Native) • L4/L7 로드밸런서 기반 수평 확장(Scale-out). • 유향 비용이 없는 서버리스 FaaS (AWS Lambda, Cloudflare Workers) 완벽 대응.
4. 네트워크 및 보안	포트 개방 불필요 (보안 격리) OS 내부 프로세스 간 통신(IPC)만 사용하므로 외부 네트워크 노출 및 인바운드 방화벽 설정 불필요.	표준 HTTPS 웹 포트 통신 외부 접근을 위한 포트 개방 및 API 키/OAuth 기반 인증 적용 필요 (단일 엔드포인트 설계로 보안 통제 수월).
5. 추천 활용 상황	네트워크 오버헤드가 없는 로컬 개발 및 단일 호스트 전용 도구 연동 시 채택.	트래픽 증감에 따라 유연한 확장이 필요한 엔터프라이즈 사내 공용 도구 마이크로서비스 구축 시 채택.

💡 **엔지니어링 선택 기준:** 단일 머신 기반 로컬 개발에는 초고속·보안 격리 장점을 가진 **Stdio**를 채택하고, 멀티 에이전트가 공유하는 분산 플랫폼이나 서버리스 환경에서는 세션 종속성이 없는 **Stateless Streamable HTTP**를 채택합니다.

FASTMCP 실습

[실습] FastMCP 기반 커스텀 도구 서버 구축

```
from fastmcp import FastMCP

# 1. FastMCP 서버 선언
mcp = FastMCP("Corporate Auditor")

# 2. 도구(Tool) 등록 - LLM이 직접 호출 가능
@mcp.tool()
def get_approval_document(doc_id: str) -> dict:
    """결재 문서 상세 내역 및 금액을 조회합니다."""
    return {"status": "success", "amount": 250000000}

# 3. 리소스(Resource) 등록 - LLM이 읽기 전용 참조
@mcp.resource("audit/policies")
def get_audit_policies() -> str:
    return "전결 규정: 2천만원 초과 건은 CISO 결재 필수"

# 4. 서버 구동 (Stdio 및 Streamable HTTP 전송 자동 지원)
mcp.run()
```

1. @mcp.tool() 데코레이터 패턴

파이썬 표준 함수를 작성하고 데코레이터를 지정하면, 함수의 Docstring과 Type Hint를 파싱하여 MCP 표준 스킴 JSON 규격으로 자동 변환하여 클라이언트에 노출합니다.

2. @mcp.resource() 데이터 바인딩

정적 가이드라인 규정 문서, 내부 템플릿, 혹은 시스템 환경 변수를 URI 포맷 (audit/policies)으로 노출하여, 에이전트가 필요 시 언제든지 읽기 전용 컨텍스트로 참조하게 바인딩합니다.

3. mcp.run() 다중 전송 계층 지원

실행 인자 분기를 통해 별도의 코드 수정 없이 로컬 CLI 프로세스 파이프(Stdio) 구동과 원격 웹서비스(Streamable HTTP) 구동을 완벽하게 동시 지원합니다.

요약 및 정리

4교시 요약: 도구 엔지니어링 및 커스텀 MCP 구축

🎯 1. 에이전트 친화적 ACI 설계

도구 명세(Docstring, Type Hint)는 LLM 추론 엔진의 핵심 컨텍스트입니다. 불필요한 마크업을 제거한 정형화된 JSON/Markdown 출력과 오류 시 실행 가능한 복구 힌트(Actionable Error Hint)를 제공해야 안정적인 자가 치유(Self-Correction)가 작동합니다.

⚡ 2. Tool Calling vs CodeAct 상호보완

정적 Tool Calling은 적은 토큰과 빠른 속도로 정형화된 작업을 저비용 처리하며, CodeAct는 샌드박스 파이썬 실행을 통해 복합 데이터 연산과 동적 유연성을 극대화합니다. 비용·속도·유연성에 맞춘 하이브리드 설계가 핵심입니다.

📖 3. 점진적 노출과 자율 스킴 순환

도구 과밀에 의한 어텐션 분산을 막기 위해 JIT 점진적 노출(Progressive Disclosure)을 적용하고, 런타임에 에이전트가 완성·검증한 코드를 재사용 가능한 지식 에셋(SKILL.md)으로 영속 저장하는 스킴 루프를 구축합니다.

🔌 4. MCP 표준 프로토콜과 FastMCP

호스트와 도구 서버의 결합을 분리하는 MCP는 최신 스펙에서 **완전한 무상태성(Stateless Core)**과 **Streamable HTTP**를 채택했습니다. 세션 핀닝 없이 클라우드 로드밸런서와 서버리스(Lambda/Workers) 환경에서 무제한 수평 확장이 가능하며, FastMCP로 이를 초고속 구현합니다.

MODULE 05

AI Agent 평가 체계와 신뢰성 엔지니어링

에이전트의 비결정성(Non-determinism)과 환각을 통제하기 위해
최종 결과물, 행동 궤적, 운영 성능을 계측하고 지속적으로 최적화하는 핵심 프레임워크

AI Agent 평가 체계 3대 축

AI 에이전트의 성능을 평가하기 위해서는 단순 텍스트 응답을 넘어 다중 턴 추론, 도구 호출 시퀀스, 운영 성능을 종합적으로 계측

🎯 1. 최종 결과물 평가 (OUTCOME)

산출물 상태 & 신뢰성 검증

에이전트가 완수한 최종 결과물의 물리적 상태 전이와 답변의 사실 정합성을 종합 진단합니다.

- ▶ **과업 상태 전이:** DB 레코드 생성, 파일 수정, Git Commit 성공 검증
- ▶ **단위 테스트:** 샌드박스 내 pytest 전체 통과(Exit Code 0) 확인
- ▶ **답변 신뢰성:** 검색 컨텍스트 기반 사실 정합성 (Faithfulness) 측정
- ▶ **보안 가드레일:** PII 마스킹, 사내 거버넌스 규정 및 안전 지침 준수

📊 대표 벤치마크: SWE-bench, GAIA, Ragas (Faithfulness)

🔑 핵심: 과업 완수율(Pass@1) · 환각 차단 · 규정 준수

🗺️ 2. 궤적 및 과정 평가 (TRAJECTORY)

도구 호출 & 추론 경로 검증

문제를 해결해 나가는 다중 턴 생각(Reasoning)과 도구 호출 시퀀스의 타당성을 검증합니다.

- ▶ **도구 선택 정밀도:** 불필요한 도구 오용(Over-tooling) 없이 최적 툴 선택
- ▶ **인자 스키마 준수:** Pydantic 스키마 타입 및 Enum 필수 인자 100% 충족
- ▶ **시퀀스 정렬:** 탐색 ➡ 인출 ➡ 변환 ➡ 검증 표준 워크플로우 준수
- ▶ **루프 차단:** 동일 도구/인자 연속 호출 무한 루프 및 턴 낭비 조기 감지

📊 대표 벤치마크: ToolBench, AgentBench, LangSmith Traces

🔑 핵심: 도구 선택 정확도 · 스키마 일치율 · 루프 0건

⚡ 3. 시스템 및 운영 성능 (OPERATIONAL)

비용 · 지연시간 · 복원력 계측

실제 서비스 상용화를 위한 토큰 비용 예산, 응답 레이턴시(TTFT), 장애 자가복구율을 계측합니다.

- ▶ **토큰 비용 예산:** 태스크당 소모 토큰 제어 및 프롬프트 캐시(90%+) 적용
- ▶ **응답 레이턴시:** 체감 반응 속도 보장을 위한 첫 토큰 시간(TTFT < 1.5s)
- ▶ **병목 프로파일링:** 외부 API 지연, DB 쿼리, LLM 추론 시간 정밀 분해
- ▶ **장애 자가 복구:** API 429 및 도구 실패 시 대체 경로 완수율(90%+)

📊 대표 모니터링: OpenTelemetry, Langfuse, Prometheus

🔑 핵심: 비용 예산 준수 · TTFT 1.5s · 복원율 90%+

에이전트 시스템의 4대 평가 대상 (Evaluation Targets)

에이전트는 단일 모델이 아닌 유기적 결합체입니다. 성능 결함이 발생했을 때 정확히 "어느 계층"을 평가하고 튜닝해야 하는지 식별



1. 시스템 프롬프트 (System Prompt)

Prompt Level

🔍 핵심 평가 영역

역할 정체성 준수, 복잡한 비즈니스 제약(Tone/Format) 충족, 환각 방지 지침 반영도

🔧 주요 튜닝 수단

힐 클라이밍(Hill Climbing), Few-shot 예시 주입, 메타 프롬프트 최적화

최적화 대상: 프롬프트 변형 (Prompt Mutation)



2. 도구 명세 & 스키마 (Tool & Schema)

Tool Level

🔍 핵심 평가 영역

Docstring 설명의 명확성(도구 오해 방지), Pydantic 인자 타입 제약, 반환 데이터 크기

🔧 주요 튜닝 수단

가공된 뷰(Curated View), 에러-as-Observation 피드백 체계

최적화 대상: 가공된 뷰 (Curated View)



3. 기저 파운데이션 모델 (Base LLM)

Model Level

🔍 핵심 평가 영역

다단계 추론(Reasoning) 능력, 함수 호출(Tool Calling) 정확도, 토큰 단가 대비 성능

🔧 주요 튜닝 수단

작업 난이도별 모델 라우팅 (Flash/Haiku vs Pro/Thinking), 모델 파인튜닝

최적화 대상: 모델 라우팅 (Model Routing)



4. 오케스트레이션 루프 & 컨텍스트 (Loop & Memory)

Harness Level

🔍 핵심 평가 영역

무한 루프 조기 차단, 컨텍스트 압축 (Compaction) 시 사실 유지율, 메모리 인출 정밀도

🔧 주요 튜닝 수단

최대 턴 제한(Circuit Breaker), Loop Breaker 미들웨어, Amnesia Guard

최적화 대상: 서킷 브레이커 (Circuit Breaker)

1. 최종 결과물 평가 (Outcome Evaluation)

사용자 또는 비즈니스 목표를 에이전트가 실제로 완수했는지 검증하기 위한 3대 핵심 결과 평가 차원

✅ 과업 성공률 (SUCCESS RATE)

Task Completion

PASS/FAIL

샌드박스 환경에서 에이전트가 완료한 파일 변경, DB 트랜잭션, API 결과물이 목표 상태를 만족하는지 검증합니다.

- ▶ **파일 생성/수정:** 지정 경로 생성 및 AST 문법 유효성 검증
- ▶ **DB 트랜잭션:** 목표 테이블 레코드 정합성 및 커밋 확인
- ▶ **테스트 스위트:** 샌드박스 내 pytest Exit Code 0 통과
- ▶ **물리적 Diff 비교:** 기대 상태(Ground Truth)와의 스냅샷 일치

🎯 **핵심 메트릭:** Pass@1, Resolve Rate (SWE-bench 표준)



검증 도구: Assertion Unit Tests (Harbor / Docker)

🔍 답변 신뢰성 & 품질

Faithfulness & Relevancy

SCORE 0.0~1.0

최종 응답이 검색 컨텍스트에 부합하는지(사실 정합성) 및 질문 의도에 적절하게 답했는지 환각 여부를 판정합니다.

- ▶ **Claim 단위 분해:** 응답 문장별 사실 진위 교차 검증
- ▶ **환각(Hallucination) 억제:** 문서 외 허위 정보 생성 선제 차단
- ▶ **의도 충실도:** 사용자 요구 제약(JSON, 표 서식, 요약) 충족
- ▶ **의미적 일관성:** 동일 질의 반복 실행 시 의미 안정도 유지

🎯 **핵심 메트릭:** Ragas Faithfulness (0.90+), Answer Relevancy



검증 도구: LLM-as-a-Judge (추론 전용 Judge 모델)

🛡️ 안전성 및 가드레일

Policy & Guardrails

CLEAN/VIOLATED

최종 결과물에 탈옥(Jailbreak) 흔적, 개인식별정보(PII) 유출, 사내 거버넌스 규정 위반 표현 포함 여부를 검사합니다.

- ▶ **PII & Key 스캔:** 주민번호, 카드번호, 사내 API 키 마스킹 검증
- ▶ **탈옥/유해성 차단:** 시스템 프롬프트 유출 및 공격 탐지
- ▶ **도메인 경계 준수:** 비즈니스 업무 외 무단 주제 이탈 방어
- ▶ **라이선스 오염 방어:** 코드 저작권 침해 및 유출 라이브러리 검사

🎯 **핵심 메트릭:** Policy Violation Rate (0.0%), Red Teaming



검증 도구: Guardrail Scanners (Llama Guard 3)

최종 결과물 평가 3대 측정 파이프라인

결과물의 물리적 상태, 논리적 사실 정합성, 정책 안전성을 계측하는 표준 4단계 측정 파이프라인

✓ 과업 성공률 측정 파이프라인 (AI 미사용 / 결정론적 유닛 테스트)

Harbor / Test Suite Harness

Step 1

목표 상태(Goal) 정의

과업 완료 후 생성/수정되어야 할 파일, DB 레코드, API 결과의 정답 기준(Ground Truth) 정의

Step 2

샌드박스(Sandbox) 실행

[Why?] 위험한 파괴적 명령으로부터 호스트를 보호하고 격리된 초기 환경(Clean Slate) 제공

Step 3

Assertion(주장) 검증

[How?] 에이전트 산출물에 대해 pytest 및 assert actual == expected 스크립트 실행

Step 4

Pass / Fail (0/1) 산출

[판정] 테스트 스크립트 Exit Code == 0 통과 시 Pass(1.0), 1개라도 실패 시 Fail(0.0)

🔍 답변 신뢰성 & 품질(Faithfulness) 측정 파이프라인 (플래그십 추론 모델 기반)

RAGAS / LLM-as-a-Judge

Step 1

골든 데이터셋 구축

테스트용 질문(Q), 원본 검색 컨텍스트(C), 전문가 작성 정답 참조본(Ground Truth) 세트 구성

Step 2

평가 루브릭(Criteria)

Faithfulness(검색 컨텍스트 외 환각 정보 포함 여부) 및 Relevancy(질문 의도 부합성) 기준표 작성

Step 3

Reasoning Judge 채점

최신 추론 전용 모델이 답변을 문장(Claim) 단위로 분해 후 컨텍스트와 사실 대조 채점

Step 4

Faithfulness Score

[산출 공식] (컨텍스트로 입증된 문장 수 / 전체 생성 문장 수) 비율로 0.0~1.0 정량 점수화

🛡️ 안전성 & 정책 가드레일 측정 파이프라인

Llama Guard / PII Scanner

Step 1

산출물 텍스트 추출

에이전트가 출력한 최종 메시지, 코드 주석, 생성 파일 텍스트 캡처

Step 2

Regex PII 규칙 스캐닝

주민등록번호, 카드번호, 사내 비밀 API 키 등 민감 패턴 1차 정규식 자동 검사

Step 3

Safety LLM 2차 심사

Llama Guard 3 모델로 탈옥(Jailbreak), 권한 오남용, 유해 표현 2차 모델 심사

Step 4

Clean vs Violated

[판정] 위반 항목 0건 시 Clean(승인), 1건 이상 감지 시 Violated(차단 및 사후 로깅)

2. 궤적 및 과정 평가 (Trajectory Evaluation)

결과뿐 아니라 문제 해결 과정의 효율성과 안전성을 검증: 적절한 도구 호출, 올바른 인자 전달, 루프 방지

도구 선택 정확도

Tool Choice Accuracy

PRECISION / RECALL

매 턴마다 문제를 해결하기 위해 가장 적절한 도구를 올바른 순서로 호출했는지 표준 경로와 비교 평가합니다.

- ▶ **도구 오용 억제:** 불필요한 도구 호출 및 불필요 토큰 비용 낭비 방지
- ▶ **최적 툴 매칭:** 다수 대안 중 가장 효율적이고 안전한 도구 우선 선택
- ▶ **시퀀스 정렬:** 탐색 ➔ 인출 ➔ 변환 ➔ 검증 표준 순서 준수
- ▶ **조기 종료 방지:** 필수 검증 단계를 누락하고 답변하는 현상 차단

 **핵심 메트릭:** Tool Selection F1-Score, Edit Distance



검증 도구: Sequence Matcher & Trajectory Evaluator

도구 인자 정밀도

Tool Arg Precision

SCHEMA MATCH %

도구를 호출할 때 LLM이 전달한 매개변수(`args`)가 사전에 정의된 Pydantic 스키마와 규칙에 부합하는지 검증합니다.

- ▶ **필수 키 누락 검사:** 필수 인자 미전달로 인한 툴 런타임 Crash 방지
- ▶ **데이터형 일치율:** 문자열, 숫자, 리스트 등 스키마 타입 100% 정합성
- ▶ **범주(Enum) 유효성:** 사전 허용된 값 범주 및 포맷 제약 엄격 준수
- ▶ **문맥 참조 정확도:** 이전 턴 도구 산출물 변수의 정확한 인자 매핑

 **핵심 메트릭:** Schema Match Rate (99%+), Zero TypeError



검증 도구: Schema Validator (Pydantic / TypeGuard)

추론 턴 효율성 & 루프 방지

Turn Efficiency

TURN & LOOP

동일 도구를 무한 호출하는 루프(Infinite Loop)나 무의미한 턴 소모 없이 최소 턴 수로 목적지에 도달했는지 검증합니다.

- ▶ **무한 루프 탐지:** 동일 도구·인자 3회 연속 중복 호출 조기 인터셉트
- ▶ **턴 효율 지수:** 최적 기준 턴(Ground Truth) 대비 실제 소요 턴 비율
- ▶ **정체(Stall) 차단:** 동일 툴 에러에 갇혀 진전이 없는 교착 상태 방어
- ▶ **자가 탈출 촉진:** 실패 시 우회 전략 및 대체 도구 즉각 시도

 **핵심 메트릭:** Step Efficiency Ratio (≤ 1.3), 0 Loop



검증 도구: Turn & Loop Detector (Circuit Breaker)

I 궤적 및 과정 평가 3대 측정 파이프라인

도구 호출 시퀀스 유사도, 인자 스키마 유효성, 무한 루프 조기 차단을 위한 4단계 실행 궤적 측정 체계

🔧 도구 선택 정확도(Tool Sequence) 측정 파이프라인

Sequence Matcher / LangSmith

Step 1

Ground Truth 시퀀스

문제 해결에 필요한 이상적인 표준 도구 호출
순서(예: Search ➔ Read ➔ Write) 사전 정의

Step 2

Trace 로그 수집

LangSmith 트레이스 로그에서 에이전트가
실제로 실행한 Tool Call 시퀀스 배열 추출

Step 3

Sequence Alignment

[알고리즘] Needleman-Wunsch 알고리즘으로
표준 순서와 실제 실행 순서 간 유사도 비교

Step 4

Precision / Recall (%)

불필요한 도구 오남용 없이 올바른 도구를
적시에 사용했는지 시퀀스 정확도(%) 산출

🧩 Pydantic Argument Schema Validator 파이프라인

Pydantic / Type Inspector

Step 1

Tool Call JSON 추출

LLM이 생성을 시도한 도구 호출 이름과 인자
파라미터(`args={"file\_path": "...}"`) 파싱

Step 2

Pydantic Schema 바인딩

[How?] 도구에 사전 정의된 `Pydantic
BaseModel` 데이터 클래스 객체로 매핑 시도

Step 3

타입/제약조건 검증

필수 인자 누락, 데이터 타입 불일치(int vs str),
허용 범주 초과 여부 자동 체크

Step 4

Schema Match Rate (%)

도구 인자 생성 중 파싱 에러/타입 오류 없이
정상 통과한 비율을 수치화 기록

🔄 추론 턴 효율성 & 루프 감지 파이프라인

Loop & Turn Counter

Step 1

매 턴 해시 로깅

매 턴 실행된 `(Tool Name + Args Hash)` 튜플을
실행 궤적 메모리에 로깅

Step 2

반복 패턴 탐지

동일한 도구와 동일한 인자의 조합이 연속 3회
이상 중복 호출되었는지 실시간 탐지

Step 3

Infinite Loop Flag

무한 루프 진입 감지 시 `Loop Flag = True` 발화
및 에이전트 루프 강제 종료

Step 4

Turn Efficiency

최종 과업 성공까지 소요된 총 턴 수 카운팅
(기준 턴 수 대비 효율성 산출)

3. 시스템 및 운영 성능 평가 (System & Operational Metrics)

상용화를 위해 반드시 통제되어야 하는 3대 비기능적 운영 지표: 비용 예산, 사용자 응답 지연시간, 장애 복원력

토큰 소모 비용 (COST)

Token Budget

COST / TOKEN

과업 완수당 소모된 Input/Output 토큰 비용을 계측하고 프롬프트 캐시 적중률을 추적하여 과도한 중복 과금을 통제합니다.

- ▶ **과업당 평균 비용:** 목표 단가(예: \$0.05/Task 이내) 엄격 준수
- ▶ **KV-Cache 적중률:** 정적 시스템 프롬프트 및 톨 스키마 90%+ 적중
- ▶ **토큰 낭비율:** 에러로 버려진 톨 및 불필요한 장문 출력 토큰 억제
- ▶ **컴팩션 절감률:** 다중 톨 컨텍스트 압축을 통한 입력 토큰 90%+ 절감

 **핵심 메트릭:** Cost per Task (\$/Task), Cache Hit Ratio (90%+)



검증 도구: Token & Cost Tracer (Langfuse / OpenLit)

응답 레이턴시 & TTFT

Latency & TTFT

TIME (sec / ms)

사용자가 첫 글자를 볼 때까지의 시간(TTFT)과 전체 과업 완수 시간(End-to-End)을 계측하고 병목 구간을 프로파일링합니다.

- ▶ **체감 반응 속도:** 첫 토큰 스트리밍 시간(TTFT < 1.5s) 보장
- ▶ **E2E 완수 SLA:** 다중 톨 전체 완료 시간(P95 < 15초) 엄격 통제
- ▶ **도구 병목 계측:** 외부 API 대기, DB 쿼리, LLM 추론 시간 정밀 분해
- ▶ **병렬 실행 최적화:** 비의존적 다중 도구 동시 실행(Fan-out) 가속

 **핵심 메트릭:** P95 TTFT (<1.5s), P95 E2E Latency, Tool Overhead %



검증 도구: Distributed Latency Tracer (OpenTelemetry)

에러 자가복구 복원율

Self-Healing Rate

RECOVERY %

도구 호출 실패, API Rate Limit(429), 컨텍스트 초과 등 장애 발생 시 대안 경로로 세션을 복구한 비율을 계측합니다.

- ▶ **API 429 지수 백오프:** Rate Limit 발생 시 자동 대기 후 완수율
- ▶ **모델 자동 폴백:** 주 모델 에러 시 백업 모델로 무중단 세션 이관
- ▶ **도구 자가 치유:** 톨 예외 반환 시 인자 수정 후 2차 시도 완수율
- ▶ **페이로드 초과 복구:** 413 에러 시 긴급 컴팩션 후 자동 재시도

 **핵심 메트릭:** Fault Recovery Rate (90%+), MTTR



검증 도구: Fault Injection Harness (Chaos Testing)

Offline Evaluation vs Inline Evaluation (배치 vs 런타임 평가)

평가 시점과 목적에 따라 배포 전 품질을 검증하는 오프라인 평가와 세션 도중 개입하는 인라인 평가로 이원화하여 운용

비교 항목	 Offline Evaluation (사후 / 배치 평가)	 Inline Evaluation (런타임 / 인루프 평가)
실행 시점 (When)	배포 전, CI/CD 파이프라인, 모델 교체 시, 야간 배치	실제 사용자 세션 처리 중, 최종 답변 반환 직전
핵심 목적 (Why)	회귀(Regression) 방지, 프롬프트 최적화, 버전 간 성능 비교	치명적 환각 차단, 보안 위반 방어, 인루프 자율 수선(Self-Correction)
지연/비용 허용도	관대함 (수 분~수 시간 배치 실행 가능, 대규모 데이터셋)	극도로 엄격함 (수 초 이내 Fast Check, 레이턴시 병목 방지)
평가 대상 (Target)	골든 데이터셋 (20~100개 대표 시나리오)	현재 세션의 실시간 입력 및 도구 궤적 (Single Run)
실패 시 조치 (Action)	배포 차단(Block Merge), 히클라이밍 루프로 프롬프트/도구 개선	모델 노드로 롤백(Rework) 후 재작업 지시 또는 안전 폴백
권장 아키텍처	배치 평가 하네스, 힐 클라이밍 옵티마이저 (app.evaluate)	가벼운 Linter ➡ Judge 미들웨어 (app.middleware.evaluator)

Inline Evaluation: 런타임 자가 교정(Self-Correction) 아키텍처

사용자에게 결함 있는 답변이 전달되기 직전(Final Answer Point), Evaluator가 개입하여 실행 가능한 피드백(Actionable Feedback)과 함께 자율 재작업 지시

Autonomous Evaluator Rework Loop

[1. Model Node] LLM 답변 생성

▼ (도구 호출 여부 감지)

도구 호출 있음 ➔ Fast Pass
(판사 생략, 지연시간 0초)

최종 답변 시점 ➔ 판사 기동
(LLM-as-a-Judge 정밀 검증)

[2. Evaluator Judge] 궤적 및 답변 감사

✓ 승인 (Approved)
사용자에게 최종 답변 반환

✗ 반려 ➔ 롤백 (Rework)
Actionable Feedback 주입 후 복귀

⚡ 1. Fast Pass: 지연시간 최소화

에이전트가 도구를 호출하는 중간 단계에는 판사를 기동하지 않습니다. 오직 사용자에게 답변을 내보내기 직전의 최종 턴에만 1회 기동하여 체감 지연시간을 획기적으로 줄입니다.

💡 2. 실행 가능한 피드백 (Actionable Feedback)

단순히 불합격 판정만 내리는 것이 아니라, "어떤 수치가 누락되었고, 어떤 도구의 결과값을 표(Table)로 요약해야 하는지" 구체적인 수정 가이드를 에이전트에게 동적으로 주입합니다.

🛑 3. 재시도 서킷 브레이커 (Rework Circuit Breaker)

에이전트가 스스로 해결하지 못하는 결함일 때 무한 재시도로 인한 API 비용 폭주와 지연을 방지하기 위해, 최대 재작업 한도(max_reworks: 1~2회) 초과 시 루프를 안전 차단합니다.

Offline Optimization: 힐 클라이밍(Hill Climbing) 탐색 원리와 루프

미분이 불가능한 텍스트 프롬프트를 평가 하네스의 점수(목적 함수)를 나침반 삼아 점진적으로 변형(Mutation)하며 최적 프롬프트 탐색

🏔️ 탐색 원리: 왜 "언덕 오르기"인가?

단 1개의 실패 사례를 고치려다 기존 99개가 깨지는 회귀(Regression)를 방지하기 위해, 전체 벤치마크 점수가 명확히 향상될 때만 한 걸음씩 산을 오릅니다.

📈 탐욕적 점수 상승 궤적 (Greedy Trajectory) 🎯 만점 도달 시 Early Stopping



[Iteration 2] 100.0점 (Global Peak 달성)

수치 집계 및 마크다운 표 명확화 지침 반영 → 최적 프롬프트 확정 & 종료

✅ Accepted



[Iteration 1] 97.5점 (성능 점진 향상)

실패 궤적 반영: "마크다운 표(Table) 서식 활용" 지침 주입 → 채택

✅ Accepted



[Iteration 0] 93.8점 (초기 베이스라인)

원시 프롬프트: "고객 문의에 친절하고 정확하게 답변하세요"

Baseline



성능 후퇴(점수 하락) 시: 산에서 내려가지 않고 즉시 이전 프롬프트로 롤백 (Rollback)

핵심 원칙: 점수가 오를 때만 한 걸음 전진(Greedy Step), 미달 시 즉시 롤백



최적화 루프: 4단계 폐루프(CLOSED-LOOP) 아키텍처

Step 1 베이스라인 벤치마크 평가 (Baseline Eval)

골든 데이터셋 N개 배치 실행 → 현재 점수($S_{current}$) 및 실패 궤적(Failure Trace) 추출

▼ 실패 사례 및 평가 피드백(Rubric Feedback) 전달

Step 2 프롬프트 제안기 (Prompt Proposer LLM)

메타 LLM이 실패 원인을 분석하여 개선된 지시문 변형(Prompt Mutation) 생성

▼ 신규 후보 프롬프트($P_{candidate}$) 배포

Step 3 배치 하네스 재평가 (Batch Re-Evaluation)

동일 골든 데이터셋 전체를 새 프롬프트로 재실행 → 신규 점수(S_{new}) 산출

▼ 점수 비교 판정 ($S_{new} > S_{current} ?$)



점수 개선: 수락 (Accept)

새 프롬프트 채택 ($P_{current} \leftarrow P_{candidate}$)



동점/하락: 반려 (Reject)

후보 즉시 폐기 & 이전 프롬프트로 롤백



루프 피드백: 목표 점수(100점) 도달 시 즉시 종료 (Early Stopping) | 미달 시 Step 2로 복귀하여 반복 탐색

실무 프롬프트 최적화 아키텍처 (Beyond Vanilla Hill Climbing)

기본 탐욕적 상승법(Vanilla)의 한계를 극복하고 엔터프라이즈 환경에서 안정적으로 프롬프트를 자동 컴파일하기 위한 핵심 기법들

🎯 1. 조기 종료 (Early Stopping)

Cost & Safety

🔍 해결 과제

만점(100점) 달성 후에도 루프가 계속 돌아 발생하는 불필요한 API 과금 및 지연 차단

🔧 실무 적용 전략

목표 점수 도달 시 즉각 루프를 종료하고 최적 프롬프트를 리턴하여 과적합 방지

핵심 원칙: 목표 달성 시 즉시 종료 (비용 절감 & 과적합 방지)

🌳 2. 빔 서치 (Beam Search Exploration)

Local Optima

🔍 해결 과제

1개 변형만 탐색하는 단순 힐클라이밍이 엉뚱한 방향의 지역 최적점(Local Optima)에 갇히는 문제

🔧 실무 적용 전략

매 턴 K개의 변형 후보를 동시에 생성하고 상위 B개의 유망 궤적을 유지하며 탐색 (DSPy 원리)

핵심 원칙: 다중 후보군 트리 탐색 (지역 최적점 극복)

📁 3. 데이터셋 격리 (Train / Val / Test)

Generalization

🔍 해결 과제

최적화에 사용된 소수 질문에만 프롬프트가 과적합되어 일반적인 사용자 쿼리가 깨지는 현상

🔧 실무 적용 전략

피드백 추출용(Train), 채택 판정용(Validation), 최종 측정용(Test)을 엄격히 분리

핵심 원칙: 데이터 분할 및 교차 검증 (일반화 성능 확보)

📏 4. 길이 페널티 (Length Penalty)

Efficiency

🔍 해결 과제

반복 최적화 과정에서 프롬프트가 수천 자로 팽창(Bloating)하여 토큰 비용과 지연시간이 폭증

🔧 실무 적용 전략

`Score = Accuracy - (Tokens × α)` 목적 함수로 간결하면서도 정확한 프롬프트 선호

핵심 원칙: 토큰 비용 및 지연시간 통제 (프롬프트 비대화 방지)

핵심 요약: 신뢰할 수 있는 에이전트를 위한 5대 평가 원칙

엔터프라이즈 환경에서 자율형 AI 에이전트를 배포하기 전에 점검해야 할 핵심 엔지니어링 체크리스트

1 TRAJECTORY & OUTCOME 동시 센싱

물리적 결과와 실행 궤적 이중 검증

텍스트 답변만 믿지 않고 물리적 상태 전이(DB/파일)와 실행 궤적(도구 호출 타당성)을 결합하여 이중 검증합니다.

- 결과 검증: DB 레코드, 파일 Diff, pytest Exit Code 0 단언
- 궤적 검증: 최적 도구 선택, Pydantic 인자 정밀도, 루프 조기 차단

🎯 핵심 검증: Docker Harbor Assertions + Sequence Matcher

2 LINTER FIRST, JUDGE SECOND

결정론적 룰 우선 평가로 비용 절감

무거운 LLM 판사 전에 정규식, Pydantic 스키마 등 결정론적 룰(Linter)을 1차 게이트로 두어 비용을 80% 절감합니다.

- 1차 결정론적 Linter: 정규식, Pydantic 스키마, 플래그 검사 (\$0 비용)
- 2차 추론적 LLM Judge: 통과된 응답에만 원자적 Claim 단위 환각 판정

🎯 핵심 검증: Pydantic Schema Validator + Ragas Faithfulness

3 BINARY APPROVAL + ACTIONABLE HINT

구조화된 피드백으로 즉각 자가교정

단순 점수가 아닌 다음 턴에 즉각 반영할 수 있는 구조화된 우회 힌트(Actionable Feedback)를 전달합니다.

- 엄격한 이진 판정: 모호한 채점 배제, 명확한 Pass/Fail 게이트 통제
- 구체적 피드백 주입: '누락된 필수 인자 포트 추가' 등 수정 가이드 제공

🎯 핵심 검증: Structured Error Schema + Actionable Feedback

4 런타임 REWORK와 서킷 브레이커

중간 턴 Fast-Pass 및 재작업 상한 통제

도구 호출 턴은 Fast Pass로 통과시키고 최종 답변에만 검증하되, 무한 롤백을 막기 위해 max_reworks로 엄격히 차단합니다.

- Fast-Pass 지연 최소화: 중간 턴 검증 부하를 배제하고 최종 산출물 시점에 집중 검증
- 자가 복구 폭주 방지: 재수행 3회 초과 시 서킷 브레이킹 발동 및 관리자(HITL) 에스컬레이션

⚡ 런타임 방어: Circuit Breaker Engine · max_reworks=3 엄격 제한

5 힐 클라이밍과 지속적 진화

운영 장애 로그의 골든 데이터셋 승격

운영 장애 Trace 로그를 골든 데이터셋으로 승격시키고, 힐 클라이밍과 Early Stopping을 통해 프롬프트를 회귀 없이 지속 진화시킵니다.

- 실패 트레이스의 자산화: 운영 장애 엡지 케이스를 골든셋으로 등록하여 회귀 테스트 자산 구축
- 지속적 평가(Hill Climbing): 프롬프트 버전 갱신 시 전체 평가셋 검증으로 성능 회귀 (Regression) 원천 방지

🌱 지속적 진화: Golden Dataset Curation · Regression Guard

MODULE 06

운영 거버넌스, 권한 통제 및 모니터링

프로덕션 에이전트의 안정성과 신뢰성 확보를 위해
3단 권한 게이트(HITL), 루프 제어, 실시간 가드레일, 샌드박스 격리 및 비동기 감사 체계를
구축합니다.

운영 및 거버넌스

운영 거버넌스 및 권한 통제 4대 핵심 체계

엔터프라이즈 에이전트의 안정적 운영을 지탱하는 4대 기둥: 권한 통제(HITL), 런타임 제어, 보안 격리, 전구간 감사 관측성

제1축 · 권한 통제

HITL 권한 게이트

파괴적 도구 실행 직전 인자를 검증하고 allow/edit/deny 3단 게이트로 인간 개입을 통제합니다.

🎯 핵심 통제 메커니즘

- ▶ 3-Tier 위계: Low/Med/High 위험도
- ▶ 인자 인터셉트: rm -rf 파싱
- ▶ 세션 인가: [y/n/a/e] 옵션 흐름

⚙️ 핵심 메커니즘

- LangGraph interrupt_before
- Checkpointer 상태 영속화
- Diff 뷰어 기반 인라인 수정

제2축 · 런타임 제어

폭주 방지 루프 제어

도구 오류 복구 과정에서 발생하는 무한 재호출 루프를 오케스트레이터 레벨에서 즉시 차단합니다.

🎯 핵심 통제 메커니즘

- ▶ 자가 복구 환각: 동일 인자 루프 차단
- ▶ 비용 폭주 방지: 초단위 토큰 소진 예방
- ▶ 안전 종료: 명확한 원인 에러 피드백

⚙️ 핵심 메커니즘

- recursion_limit=15 하드 쿼터
- 노드 재호출 스텝 카운터
- 서킷 브레이커 기반 Fail-Safe

제3축 · 보안 & 격리

가드레일 & 샌드박스

입출력 단에서 탈옥/인젝션을 차단하고, 임의 코드 실행은 일회성 격리 환경에 가둡니다.

🎯 핵심 통제 메커니즘

- ▶ 인젝션 방어: S1~S5 사전 인터셉트
- ▶ 도메인 정렬: Topic Alignment 제어
- ▶ 데이터 유출 방지: 화이트리스트 통제

⚙️ 핵심 메커니즘

- 경량 모델(gpt-4o-mini) 가드레일
- Pydantic 스키마 2단계 수선
- Docker/gVisor Ephemeral 샌드박스

제4축 · 관측성 & 감사

비동기 감사 & 관측성

추론 루프의 병목과 예산 소진을 방지하기 위해 전 구간 레이턴시와 궤적을 논블로킹 적재합니다.

🎯 핵심 통제 메커니즘

- ▶ 지연 병목 분석: 모델 vs 도구 분해
- ▶ 감사 추적성: 턴별 JSONL 영구 보관
- ▶ 비용 관리: 토큰 소진 실시간 모니터링

⚙️ 핵심 메커니즘

- AgentLogTracer 미들웨어 인터셉터
- _AsyncLogWorker 백그라운드 큐
- 턴별 레이턴시 분해 공식(T_{turn})

제1축 · 권한 통제 (PERMISSION GATE)

allow / edit / deny 3단 권한 모델과 세션 인가

작업 위험도에 따른 차등 통제와 인자(Argument) 기반 동적 인터셉트로 자율성과 안전성의 균형을 확립

3-TIER 보안 위계와 런타임 제어 메커니즘

권한 유형	해당 작업 및 리스크	하네스 런타임 제어
allow Low Risk	조회 API, 파일 검색, Read-Only 도구 호출	인간 개입 없이 자율적으로 즉각 실행 하고 결과를 컨텍스트에 갱신
edit Medium Risk	파일 생성/수정, DB 레코드 변경, 중요 인자 주입	실행을 일시정지하고 사용자가 명령·코드를 인라인 수정 하여 전송
deny High Risk	파일 삭제(rm), 외부 송출(curl), 결제 API	자율 실행을 전면 금지하며 최종 인간 승인 서명 또는 예외 강제 반환

⚠ 인자(Argument) 레벨 동적 위험 감지: bash('ls') ➔ allow, bash('rm -rf') ➔ deny

🔍 1. 인자 기반 동적 위험 인터셉터 (Argument Inspector)

도구 이름만으로 권한을 결정하면 bash, python_repl 같은 범용 도구를 제어할 수 없습니다. 에이전트가 생성한 **호출 인자**를 정적 파싱하여 rm, env(자격증명 탈취), curl(외부 스크립트 수입) 등 위험 패턴을 사전에 가로칩니다.

💡 통제 원칙: 도구 정적 화이트리스트 + 인자 정규식 Deny 인터셉터 결합

⚡ 2. Confirm 피로 방지와 세션 인가 (Claude Code 패턴)

매 작업마다 확인 팝업을 띄우면 사용자는 피로감(Confirm Fatigue)으로 무지성 Yes를 연타합니다. 이를 극복하기 위해 4단 옵션 인터랙션을 제공합니다: **[y]** 1회 승인 | **[n]** 거부 | **[a]** 세션 전체 영구 허용 | **[e]** 명령 직접 수정

💡 실무 패턴: [a] Always Allow로 신뢰 구간 내 자율성 부여 & 병목 해소

💻 3. 생산적 마찰(Productive Friction) UI/UX

단순 Yes/No 확인 대화상자 대신, 사이드 패널에 '코드 Diff 뷰어'와 '인라인 수정 에디터'를 일체형으로 제공하여 사용자가 능동적으로 코드를 검토하고 보정할 수 있게 유도합니다.

💡 UX 효과: 무지성 컨펌 방지 및 에이전트 관찰 영역으로 수정 내역 환류

제1축 · 권한 통제 (PERMISSION GATE)

LangGraph HITL 파이프라인과 3대 의사결정 (Command)

HumanInTheLoopMiddleware와 Command(resume)를 통한 도구별 정책 분기, 승인(Approve), 인자 교정(Edit), 거절 피드백(Reject) 구현

 7.Human_in_the_Loop.ipynb 핵심 구현 패턴 Command(resume) 패턴

```
# 1. 도구별 위험도 정책 분기 (Security Tiering)
hitl = HumanInTheLoopMiddleware(
    interrupt_on={
        "file_read": False,    # Tier 1 Safe: 자동 즉시 실행 (인터럽트 없음)
        "web_search": True,    # Tier 2 Review: approve, edit, reject 허용
        "file_writer": {"allowed_decisions": ["approve", "reject"]} # Tier 3: edit
    }
)
agent = create_agent(model=llm, tools=tools, middleware=[hitl],
    checkpoint=InMemorySaver())

# 2. 에이전트 실행 → __interrupt__ 페이로드 수신 및 검토
res = agent.invoke({"messages": [user_query]}, config)
req = res["__interrupt__"][0].value # action_requests(도구, 인자) & review_configs

# 3. 사람의 3대 의사결정 신호 전달 (Command 패턴)
# [옵션 A] 승인(Approve) → 원래 제안대로 실행
agent.invoke(Command(resume={"decisions": [{"type": "approve"}]}), config)

# [옵션 B] 수정(Edit) → 검색어/인자 동적 교정 실행
agent.invoke(Command(resume={"decisions": [{"type": "edit", "edited_action":
    {"name": "web_search", "args": {"query": "정제된 검색어"}}}]}), config)
```

 1. 승인 (Approve) ➡ 안전한 무손실 실행

제안된 도구와 인자가 안전하다고 판단되면 approve 결정을 전달합니다. LangGraph는 중단되었던 체크포인트 상태를 복원하여 도구를 즉시 실행하고 다음 단계로 진행합니다.

 동작 원리: Command(resume={"decisions": [{"type": "approve"}]})

 2. 인자 교정 (Edit) ➡ 할루시네이션 방어

에이전트의 검색어가 모호하거나 잘못된 경우, 사람이 edited_action으로 인자(query)를 직접 수정하여 전송합니다. 모델 재호출 없이 정확한 도구 실행을 유도합니다.

 실무 효과: 검색어 교정으로 불필요한 재귀 탐색과 API 비용 낭비 선제 차단

 3. 거절 및 피드백 (Reject) ➡ 에이전트 자가 적응

크레덴셜 파일 쓰기 등 금지 작업을 차단할 때, 거절 사유(message)를 함께 반환합니다. 에이전트는 피드백을 수용하여 파일 저장을 취소하고 마스킹된 대안 답변으로 자가 전환합니다.

 보안 핵심: 단순 Crash/중단이 아닌 가이드라인 런타임 환류(Self-adaptation)

비용 및 안정성 제어

무한 루프 감지와 오케스트레이터 차단 설계

모델 스스로 인지하지 못하는 자가 복구 환각 루프를 오케스트레이터의 서킷 브레이커로 안전 차단

루프 인터셉트 & 서킷 브레이커 파이프라인



에이전트 자가 복구 추론 (Self-Correction Loop)

도구 오류 발생 시 대안 인자를 재구성하여 재시도

▼ 동일 실패 반복 (환각 루프 진입)



루프 스텝 카운터 (Loop Step Counter)

그래프 전이 횟수 및 노드 재호출 빈도를 실시간 누적 계측

Count: 14

▼ 임계치 도달 (Threshold Reached)



서킷 브레이커 발동 (Circuit Breaker Interruption)

GraphRecursionError 트리거 ➡ 즉시 실행 중단 및 에러 반환

Limit: 15

⚠ 모델은 스스로 루프에 갇힌 것을 인지하지 못하므로, 반드시 외부 오케스트레이터가 강제 차단해야 합니다.



1. 자가 복구 실패로 인한 자원 폭주 방어

에이전트가 예외 발생 시 스스로 문제를 해결하려다 동일한 실패 유발 도구를 반복 호출하는 환각 루프에 빠질 수 있습니다. 이를 방지하면 API 토큰이 순식간에 소진되어 막대한 비용이 청구됩니다.



위험 요인: 무한 재귀 호출 방지 시 API 토큰 급속 소진 및 지연시간 급증



2. 하네스 레벨 실행 횟수 상한 제한 (Hard Quota)

상태 그래프 런타임에 recursion_limit=15와 같은 하드 리밋을 설정하여 단일 사용자 요청이 소비할 수 있는 최대 비용과 실행 시간의 상한을 확정합니다.



구현 기법: LangGraph 실행 시 config={"recursion_limit": 15} 명시적 주입



3. Fail-Safe 안전 종료 (Graceful Degradation)

서킷 브레이커가 동작하면 시스템이 비정상 다운되지 않고, 사용자에게 "지정된 복구 한도를 초과하여 안전하게 종료되었습니다"라는 명확한 원인 피드백을 전달합니다.



실무 UX: 비정상 다운 대신 정제된 안내 반환 및 관리자 모니터링 알림 전송

안전 거버넌스

실시간 입출력 가드레일(Guardrails) 파이프라인

사용자 입력부터 최종 출력까지 위협 차단, 비즈니스 정렬, 출력 스키마 자가 치유를 관장하는 심층 방어 체계

양방향 심층 방어 (DEFENSE-IN-DEPTH) 파이프라인



Input Safety & Topic Alignment (1차 필터)

S1~S5 위협(탈옥/인젝션) 차단 & 비즈니스 도메인 이탈 검사

사전 차단

▼ 검증 통과 (Safe Input)



Agent Core 추론 & 도구 실행

메인 LLM 추론 및 격리 샌드박스 내 도구 호출 수행

ReAct 실행

▼ 생성 결과 검증



Output Schema Repair & Masking (2차 필터)

PII/API 키 유출 마스킹 & Pydantic JSON 규격 2단계 수선

스키마 수선



전용 경량 모델: gpt-4o-mini (~200ms)



장애 내성: Fail-Open 정책 지원



1. Input Safety Guardrail (입력 보안)

사용자 질문이 메인 모델 추론 루프로 유입되기 전, **S4(프롬프트 인젝션 및 탈옥)**와 **S5(개인정보/API Key 탈취)**를 선제 인터셉트하여 즉각 차단 응답을 반환합니다.



사전 차단 효과: 불필요한 메인 LLM 호출 비용 절감 및 1차 보안 위협 선제 방어



2. Topic Alignment (도메인 정렬)

에이전트가 규정된 서비스 영역을 벗어나 타사 비교나 민감 논쟁에 휘말리지 않도록 통제하며, 업무 외 질의 시 수행 가능한 대안 안내(**Actionable Redirection**)를 제공합니다.



UX 가이드: 단순 거절 대신 "저는 환율 조회 도우미입니다. 환율 조회를 도와드릴까요?" 안내



3. Output Schema Repair (출력 규격 수선)

다운스트림 시스템에 전달되는 JSON이 깨졌을 때, 사소한 백틱/쉼표 오류는 **0ms** 휴리스틱 정제(\$0)로 해결하고, 필드 결함 시에만 타킷 **Re-asking**을 수행합니다.



경제성 확보: 정규식 1단계 정제로 경미한 포맷 결함을 모델 재호출 없이 즉시 복구

가드레일 아키텍처

가드레일 3대 핵심 역할 및 주요 오픈소스 생태계

엔터프라이즈 레벨 가드레일의 3대 핵심 책무(입력 보안, 도메인 정렬, 스키마 보증)와 대표 오픈소스 프레임워크 매핑

ROLE 01 · 입력 보안 게이트

Input Safety (입력 보안)

입력 텍스트의 보안 위협(탈옥, 악성코드, 시스템 지침 탈취)을 전용 분류기를 통해 사전 판정하고 차단합니다.

🔍 주요 검증 대상 및 통제 기준

- ▶ **S1~S5:** 실습 핵심 카테고리 (유해물, 인젝션, 기밀)
- ▶ **S4:** 프롬프트 인젝션 & 탈옥(Jailbreak) 방어
- ▶ **S5:** 개인정보(PII) 및 시스템 API Key 탈취 방어
- ▶ **차단 방식:** 사전 인터셉트를 통한 안전 응답 즉각 반환

대표 오픈소스 / 업계 표준

Meta Llama Guard 3

Llama Guard 14대 표준 분류 기반 경량 안전 판정

➡ 구현: **InputSafetyGuardrail**

ROLE 02 · 대화 흐름 제어

Topic Alignment (도메인 정렬)

에이전트가 지정된 비즈니스 업무 경계를 이탈하지 않도록 시맨틱 레일을 구축하고 분쟁 요소를 제어합니다.

🎯 주요 제어 메커니즘 및 UX

- ▶ **토픽 통제:** 허용(Allowed) vs 차단(Blocked) 동적 주입
- ▶ **분쟁 방어:** 타사 AI 비방 및 자극적 논쟁거리 차단
- ▶ **대안 안내:** Actionable Redirection (수행 가능 업무 제안)
- ▶ **임베딩 유사도:** FAQ/도메인 지식 앵커 기반 제어

대표 오픈소스 / 업계 표준

NVIDIA NeMo Guardrails

Colang 선언적 스크립트 기반 대화 흐름 레일 엔진

➡ 구현: **TopicAlignmentGuardrail**

ROLE 03 · 출력 규격 보증

Output Schema Repair (규격 수선)

LLM 생성 데이터가 시스템이 요구하는 엄격한 Pydantic 규격을 준수하도록 유효성을 검사하고 수선합니다.

🔧 2단계 수선 프로세스

- ▶ **Stage 1:** 마크다운 백틱/침표 0ms 휴리스틱 정제 (\$0)
- ▶ **Stage 2:** Pydantic 에러 상세 기반 타깃 Re-asking
- ▶ **안전 보증:** 다운스트림 DB 저장 실패 원천 차단
- ▶ **비용 최적화:** 정규식 1단계 처리로 경미한 결함 즉시 복구

대표 오픈소스 / 업계 표준

Guardrails AI

RAIL 스펙 기반 Pydantic 검증 및 Re-asking 라이브러리

➡ 구현: **OutputSchemaRepairGuardrail**

실행 격리 인프라

컨테이너 기반 일회성 격리 샌드박스 (Sandbox)

임의 코드 실행 및 파일 조작 도구로부터 호스트 OS를 완벽히 격리하고 실행 후 잔류물 없이 파기하는 제로-지속성 환경

격리 샌드박스의 4단계 생애주기 (EPHEMERAL LIFECYCLE)

1

Provisioning (동적 환경 할당)

요청 발생 즉시 격리된 임시 디렉토리 또는 마이크로 컨테이너 기동



2

Execution & Quota Watchdog (실행 및 감시)

CPU 1Core / Mem 512MB 제한 및 실행 타임아웃(30초) 엄격 강제



3

Safe Extraction (결과물 안전 반출)

생성된 Artifact 파일 또는 텍스트만 호스트 메모리로 복사 전달



4

Destruction (완전 파기 및 리셋)

실행 종료 즉시 컨테이너 및 임시 디렉토리를 소멸시켜 잔류물 차단



1. 호스트 파일시스템 및 OS 커널 보호

에이전트가 런타임에 작성한 파이썬 스크립트나 터미널 셸 명령어(`rm -rf`, 시스템 파일 변조)가 호스트 운영체제에 직접 도달하지 못하도록 gVisor/Docker 계층으로 격리를 보장합니다.



기술 스택: Linux Namespace / cgroups 및 gVisor 사용자 공간 커널 에뮬레이션



2. 외부 네트워크 접속 제한 (도메인 화이트리스트)

샌드박스 내부에서 임의의 외부 인터넷 접속을 전면 차단(Default Deny)하고, 사전 인가된 API 도메인만 허용하여 내부망 침투 및 기밀 데이터 유출(Exfiltration)을 방지합니다.



보안 원칙: 기본 차단(Default Deny) 정책 기반 화이트리스트 프록시 라우팅



3. 일회성 실행 환경 보장 (Ephemeral Sandbox)

에이전트 작업 완료 즉시 샌드박스를 통째로 파기함으로써, 컨테이너 내 백도어 프로세스 상주를 원천 차단합니다. (실습 8번 노트북의 *Clean-up & Reset 원리와 동일*)



청정 상태(Clean Slate): 매 태스크마다 오염되지 않은 독립 초기 파일시스템 제공

감사 로깅 및 추적

에이전트 생애주기 계측 포인트와 감사 로그 (AgentLogTracer)

미들웨어 인터셉터를 통해 에이전트의 턴 진입부터 모델 추론, 도구 실행, 턴 완료까지 전 구간을 비동기 감사 로그(JSONL)로 영구 보관

미들웨어 4대 계측 포인트 (MEASUREMENT POINTS)

P1

before_agent (턴 진입 계측)

세션 ID(thread_id), 시작 타임스탬프, 사용자 입력 질문 추출

▼ 모델 추론 호출

P2

wrap_model_call (추론 지연 및 비용 계측)

모델명, 추론 레이턴시(latency_ms), Prompt/Completion 토큰 사용량 추출

▼ 도구 실행 호출

P3

wrap_tool_call (도구 호출 인자 및 성공 계측)

도구명, 입력 인자(args), 실행 결과/에러, 소요 시간, 성공 여부 기록

▼ 턴 완료 및 반환

P4

after_agent (턴 통계 집계 및 비동기 저장)

최종 응답 및 턴 전체 통계를 _AsyncLogWorker 큐로 백그라운드 적재

```
{"event": "turn_summary",
  "session_id": "session_audit_demo_01",
  "user_query": "Nvidia GTC 2026 발표 내용 검색",
  "final_response": "NVIDIA는 GTC 2026에서...",
  "duration_ms": 1420.5,
  "token_usage": {
    "prompt_tokens": 420,
    "completion_tokens": 150,
    "total_tokens": 570
  },
  "tool_calls": [
    {
      "tool_name": "web_search",
      "arguments": {"query": "Nvidia GTC 2026"},
      "latency_ms": 310.2,
      "success": true
    }
  ],
  "timestamp": "2026-09-03T02:15:00Z"}
```

운영 관측성

에이전트 운영 2대 핵심 지표: 레이턴시 병목 및 토큰 비용

상용화를 위해 반드시 통제되어야 하는 2대 비기능적 운영 지표: 턴 전체 지연시간 분해 프로파일링과 토큰 예산 관리

턴 전체 레이턴시 분해 공식 (LATENCY BREAKDOWN)

$$T_{\text{turn}} = T_{\text{guard}} + T_{\text{LLM}} + \sum T_{\text{tools}} + T_{\text{repair}}$$

- T_{guard} (가드레일 지연): gpt-4o-mini 경량 모델 분리로 ~200ms 유지
- T_{LLM} (메인 모델 추론): 프롬프트 복합 추론 및 토큰 생성 지연
- $\sum T_{\text{tools}}$ (도구 실행): 웹 검색, 사내 DB 쿼리, 샌드박스 I/O 병목
- T_{repair} (규격 수선): 정규식 통과 시 0ms, 필드 결함 시 Re-asking 발생

AgentLogTracer 대시보드 시각화

show_timeline()과 show_tool_summary()를 통해 도구 호출 대기 시간과 LLM 추론 시간의 비중을 정량 비교하여 병목 구간을 즉각 식별합니다.

1. 토큰 예산 관리 (Cost Control)

Prompt Token vs Completion Token의 가격 차이(약 3배)와 멀티턴 누적으로 인한 프롬프트 폭증을 감시합니다. 세션당 누적 토큰 한도를 초과하면 즉시 경고하거나 인터럽트를 발생시켜 예산 폭주를 차단합니다.

💡 통제 기법: 정적 프롬프트 KV-Cache 적중률 극대화 및 멀티턴 대화 요약 압축

2. 지연시간 병목 진단 (Latency Profiling)

에이전트 응답 지연의 주원인이 "외부 도구의 네트워크 응답"인지, "LLM 추론 토큰 생성"인지를 분리 계측합니다. 도구 병목 시 캐싱(Caching) 및 비동기 프리페칭을 적용하여 사용자 체감 속도를 개선합니다.

💡 최적화 전략: TTFT 1.5초 이내 스트리밍 보장 및 외부 도구 비동기 병렬 호출

3. 가드레일 지연 오버헤드 (Guardrail Tax) 최소화

가드레일이 추가적인 레이턴시 오버헤드(Tax)가 되지 않도록, 1단계 휴리스틱 정제(0ms)와 경량 검증 모델(gpt-4o-mini)을 결합하여 총 지연시간을 200ms 이내로 억제합니다.

💡 아키텍처 원칙: 정규식 1차 게이트 통과 시 무거운 LLM 심사 스킵

관측 플랫폼

엔터프라이즈 에이전트 관측성(Observability) 플랫폼 비교

엔터프라이즈 아키텍처 환경과 보안 거버넌스 요구조건에 따른 최적의 관측성 솔루션 선정 기준

플랫폼	핵심 기능 및 관측 특성	실무 최적 적용 시나리오
LangSmith SAAS / 클라우드	LangGraph 상태 그래프 노드 전이, 조건부 분기, 체크포인트 메모리 상태 및 도구 입출력 궤적을 1:1로 정밀 시각화	LangGraph 기반의 복잡한 멀티에이전트 추론 루프 및 힐클라이밍 평가 최적화 단계
Arize Phoenix OPEN-SOURCE / 온프레미스	온프레미스/로컬 컨테이너 기반으로 기동 가능하며, 임베딩 벡터 공간 시각화 및 에이전트 추론 결과의 Auto-Eval 자동 평가 기능 내장	사내 기밀 데이터 유출 방지를 위해 외부 SaaS 전송 없이 사내 폐쇄망에서 관측성을 확보해야 하는 환경
Langfuse OPEN-SOURCE / HYBRID	가벼운 SDK 연동을 통한 분산 트레이싱(Trace), 프롬프트 버전 릴리즈 관리, 토큰 비용 실시간 대시보드 및 지연시간 통계 분석 제공	비용 효율성과 프롬프트 버전 거버넌스를 결합하여 운영 서비스의 전방위 모니터링을 구축하려는 팀
AgentLogTracer 경량 로컬 / 무의존성	별도 외부 서버 구축 없이, _AsyncLogWorker 백그라운드 비동기 큐를 통해 세션/턴 계층 JSONL 감사 로그를 오버헤드 없이 디스크에 저장	단위 테스트, 로컬 주피터 실행 및 네트워크 의존성 없는 격리 샌드박스 내부 자체 로깅 파이프라인

핵심 정리

운영 거버넌스 및 권한 통제 4대 원칙 요약

신뢰할 수 있는 엔터프라이즈 AI 에이전트를 프로덕션에 배포하기 전에 반드시 검증해야 할 4대 아키텍처 원칙

원칙 01 · 권한 통제 & 런타임 제어

3단 권한 게이트 & 무한 루프 차단

위험 인자 파싱 기반 allow/edit/deny 게이트와 LangGraph 영속 인터럽트를 구축하고, recursion_limit로 자가 복구 환각 폭주를 차단합니다.

- [y/n/a/e] 세션 인가 및 Diff 뷰어 기반 인라인 수정 UX
- 서킷 브레이커(Circuit Breaker) 기반 Fail-Safe 안전 종료

🔑 핵심 체크포인트: interrupt_before 영속 인터럽트 & recursion_limit

원칙 02 · 안전 거버넌스

3대 실시간 입출력 가드레일 & 경량 모델 분리

입력 보안(S1~S5, 탈옥 방어), Topic Alignment (도메인 정렬), 2단계 스키마 수선을 적용하며, 가드레일 전용 경량 모델(gpt-4o-mini)을 사용하여 지연을 억제합니다.

- 0ms 휴리스틱 정제와 타깃 Re-asking 결합으로 지연 오버헤드 최소화
- 사전 차단 게이트로 메인 고비용 LLM 토큰 낭비 선제 방어

🔑 핵심 체크포인트: S1~S5 가드레일 & Fail-Open 정책 결합

원칙 03 · 실행 격리

일회성 격리 샌드박스 (Ephemeral Sandbox)

도구의 파일 생성 및 임의 코드 실행은 호스트와 격리된 환경에서 기동하고 작업 완료 즉시 파기(Zero-Persistence)하여 백도어 상주와 시스템 변조를 원천 차단합니다.

- CPU/Mem 리소스 감시 및 실행 타임아웃 엄격 강제
- 외부 네트워크 접속 제한(도메인 화이트리스트)으로 데이터 유출 방지

🔑 핵심 체크포인트: Ephemeral Lifecycle & Clean Slate 제공

원칙 04 · 감사 관측성

생애주기 계측 기반 비동기 감사 로그

before_agent, wrap_model_call, wrap_tool_call을 통해 턴별 레이턴시, 토큰, 도구 호출 파라미터를 메인 루프 지연 없이 백그라운드 큐로 저장합니다.

- 세션/턴 계층 JSONL 감사 로그로 사후 이상 징후 완벽 추적
- 턴별 레이턴시 분해 공식 기반 병목 구간 즉각 식별

🔑 핵심 체크포인트: _AsyncLogWorker 비동기 적재 & 비용 분석

COURSE COMPLETED — AGENT HARNESS ENGINEERING

Thank You!

교육에 참여해 주셔서 대단히 수고하셨습니다.

비결정적 LLM을 결정론적 비즈니스 프로세스로 제어하는 실전 엔지니어링 역량을 바탕으로,
안정적이고 신뢰할 수 있는 엔터프라이즈 프로덕션 에이전트를 성공적으로 구축하시길 응원합니다.

CORE LOOP

결정론적 실행 하네스

ReAct 한계를 극복하는 4대 조율 장치와 계획·복구
아키텍처

CONTEXT & MEMORY

4계층 영속 기억 체계

5계층 프롬프트 조립, JIT 일화 회상 및 5단계 컴팩
션 방어선

RELIABILITY & GOVERNANCE

신뢰성 평가 & 권한 통제

HITL 3단 게이트, 샌드박스 격리 및 힐클라이밍 최
적화

INSTRUCTOR

김형욱 강사

CONTACT

hukim@artiasolution.com